

**ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ VIỄN THÔNG
BỘ MÔN MÁY TÍNH – HỆ THỐNG NHÚNG**



HỒNG QUỐC BẢO

Đề tài khóa luận tốt nghiệp:

**THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG
GATEWAY CAN-ETHERNET THEO KIẾN
TRÚC PHÂN TẦNG ỨNG DỤNG TRONG
CHẨN ĐOÁN Ô TÔ**

Chuyên ngành Máy Tính - Hệ Thống Nhúng

TP. Hồ Chí Minh, tháng 08 năm 2026

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA ĐIỆN TỬ - VIỆN THÔNG
BỘ MÔN MÁY TÍNH - HỆ THỐNG NHÚNG



HỒNG QUỐC BẢO
22200011

Đề tài:

THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG
GATEWAY CAN-ETHERNET THEO KIẾN
TRÚC PHÂN TẦNG ỨNG DỤNG TRONG
CHẨN ĐOÁN Ô TÔ

DESIGN AND IMPLEMENTATION OF A CAN-
ETHERNET GATEWAY SYSTEM USING A LAYERED
ARCHITECTURE FOR AUTOMOTIVE DIAGNOSTICS

KHÓA LUẬN TỐT NGHIỆP CỬ NHÂN
NGÀNH KỸ THUẬT ĐIỆN TỬ - VIỆN THÔNG
CHUYÊN NGÀNH MÁY TÍNH - HỆ THỐNG NHÚNG

NGƯỜI HƯỚNG DẪN KHOA HỌC
ThS. HOÀNG ANH TUẤN

TP. Hồ Chí Minh, tháng 08 năm 2026

TÊN ĐỀ TÀI:

**THIẾT KẾ VÀ TRIỂN KHAI HỆ THỐNG GATEWAY
CAN-ETHERNET THEO KIẾN TRÚC PHÂN TẦNG ỨNG DỤNG
TRONG CHẨN ĐOÁN Ô TÔ**

*DESIGN AND IMPLEMENTATION OF A CAN-ETHERNET GATEWAY SYSTEM USING A LAYERED
ARCHITECTURE FOR AUTOMOTIVE DIAGNOSTICS*

Sinh viên: Hồng Quốc Bảo

Người hướng dẫn: ThS. Hoàng Anh Tuấn

Hội đồng đánh giá:

- 1. **Chủ tịch:** xác nhận:
- 2. **Thư ký:** xác nhận:
- 3. **Ủy viên:**
- 4. **Ủy viên:**
- 5. **Ủy viên:**

LỜI CAM KẾT

Tôi xin cam đoan những số liệu và kết quả nghiên cứu được trình bày trong khóa luận tốt nghiệp là trung thực. Tôi cam kết không sao chép nội dung hay kết quả của các nghiên cứu khác đã được công bố. Những tài liệu tham khảo được tôi sử dụng trong báo cáo đã được trích dẫn một cách đầy đủ, rõ ràng về nguồn gốc theo quy định.

Tôi xin chịu trách nhiệm nếu vi phạm các cam kết trên.

TP. Hồ Chí Minh, tháng 8 năm 2026

Sinh viên thực hiện

Hồng Quốc Bảo

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện khóa luận tốt nghiệp với đề tài “Thiết kế và triển khai hệ thống Gateway CAN-Ethernet theo kiến trúc phân tầng ứng dụng trong chẩn đoán ô tô”, em đã nhận được rất nhiều sự quan tâm, giúp đỡ và động viên từ quý Thầy Cô, gia đình và bạn bè.

Trước tiên, em xin gửi lời tri ân chân thành và sâu sắc nhất đến ThS. Hoàng Anh Tuấn. Thầy đã dành rất nhiều thời gian, tâm huyết để tận tình hướng dẫn, định hướng nghiên cứu và giải đáp những thắc mắc kỹ thuật phức tạp về mạng viễn thông trên ô tô cho em. Những lời khuyên, sự chỉ bảo tận tình và những yêu cầu khắt khe từ Thầy chính là động lực to lớn giúp em hoàn thiện được hệ thống Gateway và cuốn báo cáo này.

Em xin trân trọng cảm ơn Ban Giám hiệu Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM, cùng toàn thể quý Thầy Cô trong Khoa Điện tử - Viễn thông, đặc biệt là các Thầy Cô thuộc Bộ môn Máy tính - Hệ thống nhúng. Những kiến thức nền tảng vững chắc mà các Thầy Cô đã truyền đạt trong suốt 4 năm giảng đường chính là hành trang quý giá nhất để em có thể tiếp cận và giải quyết các bài toán thực tế.

Em cũng xin dành tình cảm biết ơn sâu sắc nhất đến gia đình, những người đã luôn là hậu phương vững chắc, hy sinh và tạo mọi điều kiện tốt nhất để em yên tâm theo đuổi đam mê kỹ thuật. Xin cảm ơn những người bạn đồng trang lứa đã luôn sát cánh, cùng nhau chia sẻ áp lực học tập và đồng hành trong suốt chặng đường đại học vừa qua.

Mặc dù đã nỗ lực hết sức mình, nhưng do giới hạn về mặt thời gian cũng như kinh nghiệm thực tế chưa nhiều, cuốn khóa luận chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự thông cảm, cũng như những lời góp ý quý báu từ quý Thầy Cô trong Hội đồng đánh giá để đề tài được hoàn thiện hơn và bản thân em có thêm góc nhìn sâu sắc cho con đường sự nghiệp sau này.

Em xin chân thành cảm ơn!

TP. Hồ Chí Minh, tháng 08 năm 2026

Sinh viên thực hiện

Hồng Quốc Bảo

MỤC LỤC

LỜI CAM KẾT	i
LỜI CẢM ƠN	iii
TÓM TẮT KHÓA LUẬN	ix
DANH MỤC TỪ VIẾT TẮT	xi
DANH SÁCH HÌNH	xiii
DANH SÁCH BẢNG	xv
1 GIỚI THIỆU ĐỀ TÀI	1
1.1 Bối cảnh của đề tài	1
1.2 Lý do thực hiện đề tài	1
1.3 Mục tiêu của đề tài	2
1.4 Phạm vi và giới hạn của đề tài	2
1.4.1 Giới hạn về phần cứng và tiêu chuẩn môi trường	2
1.4.2 Giới hạn về chuẩn hạ tầng truyền thông vật lý	3
1.4.3 Giới hạn về an toàn thông tin	3
1.4.4 Giới hạn về tập dịch vụ chẩn đoán UDS	3
1.5 Bố cục của khóa luận	3
2 CƠ SỞ LÝ THUYẾT	5
2.1 Mạng Controller Area Network (CAN)	5
2.1.1 Cấu trúc chung của khung truyền dữ liệu mạng CAN	5
2.1.2 Kiến trúc bộ điều khiển CAN trên vi điều khiển STM32	6
2.1.3 Cơ chế định tuyến chẩn đoán với Standard ID	7
2.2 Giao thức chẩn đoán hợp nhất UDS (ISO 14229)	7
2.2.1 Ứng dụng thực tiễn và Kiến trúc tham chiếu OSI	8
2.2.2 Kiến trúc thông điệp Yêu cầu và Phản hồi UDS	9
2.2.3 Phân tích cấu trúc các dịch vụ UDS trong phạm vi nghiên cứu . .	10
2.3 Các giao thức tầng vận chuyển mạng ô tô	11
2.3.1 Giao thức vận chuyển qua mạng CAN (ISO-TP / ISO 15765-2) .	11

2.3.2	Giao thức chẩn đoán qua nền tảng Internet Protocol (DoIP / ISO 13400)	12
2.3.3	Kiến trúc Phần mềm nhúng đa nhiệm FreeRTOS và LwIP	13
2.3.4	Kiến trúc phân tầng phần mềm	14
2.3.5	Vòng đời và cơ chế chuyển đổi trạng thái của luồng tác vụ	15
2.3.6	Ngăn xếp mạng rút gọn LwIP trong hệ thống nhúng	16
3	THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG	17
3.1	Tổng quan kiến trúc hệ thống	17
3.2	Thiết kế phần cứng hệ thống	18
3.2.1	Lựa chọn vi điều khiển trung tâm	18
3.2.2	Thiết kế mạch giao tiếp mạng CAN	19
3.2.3	Thiết kế mạch giao tiếp Ethernet (PHY)	20
3.2.4	Thiết kế khối thu thập dữ liệu Node chẩn đoán	21
3.3	Thiết kế phần mềm và thuật toán điều khiển	22
3.3.1	Kiến trúc phần mềm tổng thể	22
3.3.2	Kiến trúc đa nhiệm với FreeRTOS	23
3.3.3	Thuật toán điều khiển node chẩn đoán (STM32F1)	25
3.3.4	Thuật toán định tuyến Gateway (STM32F4)	27
3.4	Thiết kế giao diện người dùng và chức năng	28
3.5	Cấu trúc bản tin chẩn đoán UDS và DoIP	30
3.5.1	Cấu trúc chung của khung truyền DoIP	30
3.5.2	Đóng gói bản tin ứng dụng UDS	30
3.5.3	Cơ chế tịnh tiến giao thức tại Gateway	31
3.6	Phương pháp đo lường và đánh giá hiệu năng hệ thống	31
3.6.1	Kỹ thuật đo lường độ trễ cấp vi mô thông qua thanh ghi DWT	32
3.6.2	Đánh giá độ trễ toàn trình dưới điều kiện tải cao	32
4	KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ	33
4.1	Môi trường và thiết bị thực nghiệm	33
4.1.1	Cấu hình phần cứng	33
4.1.2	Công cụ phần mềm	34
4.2	Kiểm thử chức năng và chuẩn hóa giao thức	35
4.2.1	Xác minh gói tin trên Wireshark	35
4.2.2	Kiểm thử các dịch vụ chẩn đoán UDS cốt lõi	36
4.3	Đánh giá hiệu năng độ trễ hệ thống	37

4.3.1	Kết quả đo lường độ trễ định tuyến cấp vi mô	37
4.3.2	Đánh giá độ trễ toàn trình RTT và kiểm thử độ ổn định	38
4.4	Trích xuất và lưu trữ dữ liệu hệ thống	41
5	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	43
5.1	Kết luận chung	43
5.1.1	Đóng góp của đề tài	43
5.1.2	Kiến thức và kỹ năng đạt được	43
5.2	Hạn chế của đề tài	44
5.3	Hướng phát triển	44
	TÀI LIỆU THAM KHẢO	45

TÓM TẮT KHÓA LUẬN

Khóa luận nghiên cứu và triển khai hệ thống Automotive Gateway đa giao thức nhằm kết nối mạng CAN truyền thống và Ethernet tốc độ cao trên nền tảng vi điều khiển STM32. Phần mềm được thiết kế theo kiến trúc phân tầng, bao gồm lớp MCAL, lớp Middleware (tích hợp FreeRTOS và ngăn xếp LwIP) và lớp Ứng dụng. Dữ liệu cảm biến được lọc nhiễu qua bộ lọc số Exponential Smoothing trước khi truyền. Hệ thống tích hợp các tiêu chuẩn chẩn đoán DoIP và UDS (ISO 14229) hỗ trợ các dịch vụ như đọc dữ liệu thời gian thực (0x22), khởi động lại ECU (0x11) và điều khiển cơ cấu chấp hành (0x2F). Giao diện giám sát trên máy tính viết bằng Python (CustomTkinter) cho phép trực quan hóa dữ liệu, ghi log ra file CSV và gửi lệnh chẩn đoán. Kết quả thực nghiệm chứng minh hệ thống hoạt động ổn định, đạt độ trễ thấp và đáp ứng yêu cầu thời gian thực.

THESIS ABSTRACT

DESIGN AND IMPLEMENTATION OF A CAN-ETHERNET GATEWAY SYSTEM USING A LAYERED ARCHITECTURE FOR AUTOMOTIVE DIAGNOSTICS

This thesis researches and implements a multi-protocol Automotive Gateway bridging traditional CAN and high-speed Ethernet networks on the STM32 microcontroller. The software follows a standard layered architecture, encompassing MCAL, Middleware (integrating FreeRTOS and LwIP), and Application layers. Sensor data is filtered via an Exponential Smoothing digital filter prior to transmission. The system implements DoIP and UDS protocols (ISO 14229), supporting core services such as real-time data reading (0x22), ECU Reset (0x11), and actuator control (0x2F). A Python-based PC monitoring interface (CustomTkinter) provides graphical visualization, CSV data logging, and direct command transmission. Experimental results demonstrate low routing latency and compliance with real-time requirements.

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
ADAS	Advanced Driver Assistance Systems	Hệ thống hỗ trợ người lái nâng cao
ADC	Analog-to-Digital Converter	Bộ chuyển đổi tương tự - số
API	Application Programming Interface	Giao diện lập trình ứng dụng
ARM	Advanced RISC Machines	Kiến trúc vi xử lý RISC nâng cao
CAN	Controller Area Network	Mạng điều khiển vùng
CAN-FD	Controller Area Network Flexible Data-rate	Mạng bộ điều khiển vùng với tốc độ dữ liệu linh hoạt
CPU	Central Processing Unit	Bộ xử lý trung tâm
DID	Data Identifier	Mã định danh dữ liệu
DoCAN	Diagnostics over Controller Area Network	Chẩn đoán qua mạng CAN
DoIP	Diagnostic communication over IP	Giao tiếp chẩn đoán qua nền tảng IP
ECU	Electronic Control Unit	Bộ điều khiển điện tử
EMC	Electromagnetic Compatibility	Tương thích điện từ
EMI	Electromagnetic Interference	Nhiều điện từ
GUI	Graphical User Interface	Giao diện người dùng đồ họa
HAL	Hardware Abstraction Layer	Lớp thư viện trừu tượng phần cứng
ISO-TP	ISO Transport Protocol	Giao thức vận chuyển ISO 15765-2
LwIP	Lightweight IP	Ngăn xếp giao thức IP rút gọn
MAC	Media Access Control	Bộ điều khiển truy cập môi trường
MCAL	Microcontroller Abstraction Layer	Lớp trừu tượng vi điều khiển
NRC	Negative Response Code	Mã lỗi phản hồi tiêu cực

Viết tắt	Tiếng Anh	Tiếng Việt
OEM	Original Equipment Manufacturer	Nhà sản xuất thiết bị gốc
OS	Operating System	Hệ điều hành
OSI	Open Systems Interconnection	Mô hình tham chiếu kết nối các hệ thống mở
PCB	Printed Circuit Board	Bảng mạch in
PDU	Protocol Data Unit	Đơn vị dữ liệu giao thức
PHY	Physical Layer Transceiver	IC thu phát tầng vật lý
RMII	Reduced Media Independent Interface	Giao diện truyền thông độc lập môi trường rút gọn
RTOS	Real-Time Operating System	Hệ điều hành thời gian thực
RTT	Round-Trip Time	Thời gian vòng (từ lúc phát lệnh đến khi nhận kết quả)
SID	Service Identifier	Mã định danh dịch vụ
TCP/IP	Transmission Control Protocol / Internet Protocol	Giao thức điều khiển truyền nhận / Giao thức liên mạng
UDP	User Datagram Protocol	Giao thức gói dữ liệu người dùng
UDS	Unified Diagnostic Services	Các dịch vụ chẩn đoán hợp nhất
VIN	Vehicle Identification Number	Số nhận dạng xe

DANH SÁCH HÌNH

2.1	Cấu trúc chung của khung truyền dữ liệu mạng CAN	6
2.2	Sơ đồ khối chức năng của ngoại vi điều khiển CAN trên vi điều khiển STM32	7
2.3	Ánh xạ kiến trúc giao thức UDS, DoCAN và DoIP trên mô hình tham chiếu OSI	8
2.4	Cấu trúc ánh xạ và lưu đồ phân mảnh gói tin theo tiêu chuẩn ISO-TP . .	12
2.5	Kiến trúc phân tầng phần mềm hệ thống nhúng với RTOS	14
2.6	Sơ đồ chuyển đổi trạng thái của một luồng tác vụ trong FreeRTOS	15
3.1	Sơ đồ khối tổng thể kiến trúc hệ thống Gateway CAN-Ethernet	18
3.2	Sơ đồ nguyên lý mạch giao tiếp CAN sử dụng IC TJA1050	19
3.3	Sơ đồ nguyên lý giao tiếp RMII và cấu hình IC Ethernet LAN8720A . .	20
3.4	Sơ đồ nguyên lý kết nối giữa node A và cảm biến siêu âm HC-SR04 . . .	21
3.5	Sơ đồ kiến trúc phần mềm 3 lớp (MCAL, Middleware, Application) của hệ thống	23
3.6	Lưu đồ thuật toán điều khiển chương trình chính tại node STM32F1 . . .	25
3.7	Lưu đồ chi tiết thuật toán lọc số học và phân loại ngưỡng cảnh báo	26
3.8	Lưu đồ thuật toán định tuyến chuyển đổi giao thức CAN - Ethernet DoIP tại Gateway	28
3.9	Giao diện phần mềm chẩn đoán UDS Client trên nền tảng PC	29
4.1	Mô hình kết nối phần cứng thực nghiệm hệ thống Gateway DoIP/CAN .	34
4.2	Trích xuất gói tin Request từ phía máy tính Tester qua bộ lọc Wireshark .	35
4.3	Gói tin Reply phản hồi từ Gateway về máy tính Tester	35
4.4	Log UART minh họa quá trình thực thi dịch vụ 0x2F với chức năng Short Term Adjustment, thay đổi các mức tín hiệu chấp hành tuần tự từ Mode 0 đến Mode 3 và lệnh trả về chế độ tự động AUTO Mode	36
4.5	Log UART ghi nhận toàn bộ quá trình xử lý dịch vụ 0x11, bao gồm giải mã gói tin DoIP, chuyển tiếp lệnh khởi động lại xuống Node mạng và xử lý gói tin phản hồi mang trạng thái chẩn đoán từ ECU đích	36
4.6	Log UART thể hiện kết quả thực thi dịch vụ 0x22 để truy vấn dữ liệu cảm biến siêu âm, minh họa cơ chế tối ưu thông qua Cache Hit giúp trả về tức thời các giá trị khoảng cách đo lường như 130 cm và 132 cm . . .	37

4.7	Thông số đo lường độ trễ định tuyến micro-latency qua khối DWT	38
4.8	Biểu đồ lưu lượng gói tin Ethernet trên Wireshark trong kịch bản kiểm thử hoạt động liên tục	38
4.9	Bảng chứng đo lường độ trễ toàn trình RTT của các dịch vụ UDS trên Wireshark	39
4.10	Dữ liệu chẩn đoán UDS và thông số độ trễ được trích xuất thành file CSV	41

DANH SÁCH BẢNG

2.1	Bảng tra cứu mã lỗi tiêu cực (NRC) phổ biến	9
2.2	Đặc tả cấu trúc cấu thành 8 byte DoIP Header chuẩn ISO 13400	12
3.1	Bảng sơ đồ kết nối phần cứng giữa vi điều khiển (F4/F1) và IC thu phát CAN TJA1050	19
3.2	Bảng sơ đồ kết nối giao diện RMII giữa STM32F4 và IC LAN8720A . .	20
3.3	Bảng sơ đồ kết nối phần cứng giữa Node STM32F1 và module Cảm biến siêu âm	21
3.4	Cấu hình các luồng tác vụ trong FreeRTOS	24
4.1	Kết quả định lượng độ trễ toàn trình RTT phân loại theo dịch vụ UDS . .	40

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh của đề tài

Trong kỷ nguyên công nghiệp 4.0, ngành công nghiệp ô tô đang trải qua một quá trình chuyển dịch mạnh mẽ từ những cỗ máy cơ khí truyền thống sang định hướng “Xe định nghĩa bằng phần mềm”. Trên các dòng xe hiện đại, số lượng các ECU ngày càng gia tăng nhằm phục vụ cho hệ thống ADAS, hệ thống thông tin giải trí và xe tự hành.

Sự gia tăng nhanh chóng về số lượng ECU kéo theo sự mở rộng theo cấp số nhân của lưu lượng dữ liệu. Mạng CAN truyền thống, mặc dù có độ tin cậy cao và chi phí thấp, nhưng với băng thông giới hạn (tối đa 1 Mbps) đang dần bộc lộ những điểm nghẽn vật lý. Để giải quyết bài toán này, mạng Automotive Ethernet với băng thông lớn (từ 100 Mbps đến hàng Gbps) đang được đưa vào làm mạng xương sống [5]. Tuy nhiên, việc thay thế hoàn toàn mạng CAN là bất khả thi trong tương lai gần do tính ưu việt của nó trong việc điều khiển các cơ cấu chấp hành yêu cầu tính thời gian thực khắt khe. Do đó, một kiến trúc lai kết hợp giữa CAN và Ethernet đang trở thành tiêu chuẩn tất yếu của ngành công nghiệp [8,9]. Trong kiến trúc này, một thiết bị trung gian đóng vai trò chuyển đổi và định tuyến giao thức – Automotive Gateway – là thành phần bắt buộc phải có để luân chuyển dữ liệu xuyên suốt giữa hai môi trường mạng khác biệt.

1.2 Lý do thực hiện đề tài

Nhận thấy tầm quan trọng cốt lõi của Automotive Gateway trong kiến trúc xe hơi thế hệ mới, cùng với định hướng áp dụng các kiến thức chuyên sâu về hệ thống nhúng, vi điều khiển ARM Cortex-M [3] và mạng truyền thông vào một bài toán thực tiễn, đề tài “Thiết kế và triển khai hệ thống Gateway CAN-Ethernet theo kiến trúc phân tầng ứng dụng trong chẩn đoán ô tô” đã được lựa chọn để thực hiện.

Việc áp dụng phương pháp thiết kế phân tầng từ phần cứng (MCAL) đến phần mềm (Middleware, Application) không chỉ giúp hệ thống tối ưu hóa tài nguyên mà còn đảm bảo tính module hóa, dễ dàng mở rộng và bảo trì. Bên cạnh nhiệm vụ định tuyến gói tin thông thường, ngành ô tô hiện nay đang tiêu chuẩn hóa việc chẩn đoán lỗi từ xa thông qua giao thức DoIP (ISO 13400) [2] kết hợp với các dịch vụ chẩn đoán hợp nhất UDS (ISO 14229) [1]. Việc hiện thực hóa ngăn xếp giao thức này trên vi điều khiển không chỉ củng cố nền tảng kỹ thuật mà còn là bước chuẩn bị quan trọng nhằm đáp ứng quy trình phát triển phần mềm ô tô chuyên nghiệp hiện nay.

1.3 Mục tiêu của đề tài

Đề tài hướng tới việc thiết kế và lập trình một hệ thống Automotive Gateway đa giao thức hoàn chỉnh [9]. Hệ thống được phát triển trên vi điều khiển ARM Cortex-M4 [3], tích hợp hệ điều hành FreeRTOS [4] và LwIP. Mã nguồn được cấu trúc theo tiêu chuẩn phân tầng công nghiệp (MCAL, Middleware, Application) nhằm thiết lập luồng định tuyến hai chiều tối ưu giữa mạng CAN và Ethernet (UDP/IPv4) [8].

Về mặt truyền thông và chẩn đoán, đề tài xây dựng node cảm biến tích hợp bộ lọc số Exponential Smoothing trực tiếp trên phần cứng để giảm nhiễu trước khi phát lên mạng CAN. Đồng thời, Gateway trung tâm đảm nhiệm việc hiện thực hóa tiêu chuẩn DoIP [2] và hỗ trợ các dịch vụ chẩn đoán UDS [1] cốt lõi bao gồm *ECU Reset (0x11)*, *Read Data By Identifier (0x22)* và *IO Control By Identifier (0x2F)*.

Nhằm mục đích tương tác và đánh giá, một ứng dụng giám sát bằng Python [7] được phát triển để theo dõi dữ liệu và phát lệnh UDS trực quan. Cuối cùng, năng lực vận hành của hệ thống được kiểm chứng khách quan bằng phần mềm Wireshark và các kịch bản Stress Test trên mạng CAN. Qua đó, đề tài có thể đo lường tổng thời gian từ lúc phát lệnh đến khi nhận kết quả (RTT) [10], đồng thời kiểm chứng khả năng lọc dữ liệu ổn định của Gateway khi lưu lượng mạng bị quá tải.

1.4 Phạm vi và giới hạn của đề tài

Do giới hạn quỹ thời gian và tài nguyên phần cứng, khóa luận tập trung giải quyết các bài toán nền tảng về thiết kế kiến trúc phần mềm và định tuyến giao thức. Để đảm bảo tính khả thi, hệ thống được giới hạn trong các phạm vi cụ thể nhằm tối ưu quá trình đánh giá hiệu năng thay vì triển khai một sản phẩm thương mại hoàn chỉnh:

1.4.1 Giới hạn về phần cứng và tiêu chuẩn môi trường

Hệ thống hiện tại sử dụng các bo mạch phát triển đa dụng để xây dựng nguyên mẫu. Quá trình kết nối vật lý chủ yếu dựa trên hệ thống dây cắm linh hoạt nhằm phục vụ gỡ lỗi và phân tích tín hiệu trong phòng thí nghiệm. Do đó, khóa luận chưa đi sâu thiết kế mạch PCB chống nhiễu chuyên nghiệp, dẫn đến việc hệ thống chưa đáp ứng các tiêu chuẩn tương thích điện từ EMC và nhiễu điện từ EMI khắt khe, cũng như chưa được đánh giá qua bài kiểm tra rung lắc hay nhiệt độ khắc nghiệt trong khoang động cơ ô tô thực tế.

1.4.2 Giới hạn về chuẩn hạ tầng truyền thông vật lý

Đối với hạ tầng giao tiếp, đề tài triển khai mạng CAN 2.0 cơ bản (tốc độ tối đa 1 Mbps). Do giới hạn phần cứng, hệ thống chưa mở rộng lên CAN-FD để tăng băng thông. Đồng thời, liên kết Ethernet giữa Gateway và Tester đang dùng chuẩn dân dụng 100BASE-TX qua cáp RJ45, thay vì chuẩn Automotive Ethernet 100BASE-T1 trên cáp xoắn đơn chuyên dụng của xe hơi [5]. Tuy nhiên, sự tinh gọn phần cứng này không ảnh hưởng đến bản chất định tuyến và tính toàn vẹn của gói tin logic ở các tầng mạng cao hơn.

1.4.3 Giới hạn về an toàn thông tin

Trọng tâm của đề tài là chứng minh tính đúng đắn của giải thuật định tuyến đa mạng và chu trình bọc, mở gói bản tin [9]. Do đó, kiến trúc Gateway ưu tiên tài nguyên CPU để tối ưu tốc độ và giảm độ trễ toàn trình. Hệ thống chưa tích hợp các cơ chế Cybersecurity tiên tiến như mã hóa đường truyền, chứng chỉ số hay xác thực thông điệp MAC. Việc thiếu vắng các lớp bảo mật này khiến nguyên mẫu chưa thể phòng vệ trước các tấn công mạng như nghe lén, giả mạo nút hay Replay Attack.

1.4.4 Giới hạn về tập dịch vụ chẩn đoán UDS

Giao thức UDS [1] được hiện thực hóa để làm tải trọng kiểm chứng chu trình bọc và mở gói của chuẩn DoIP [2]. Thay vì bao phủ toàn bộ tập lệnh chẩn đoán đồ sộ, khóa luận chỉ tập trung hoàn thiện luồng vận hành cốt lõi của các dịch vụ tiêu biểu. Cụ thể, hệ thống triển khai thành công ECU Reset 0x11, Read Data By Identifier 0x22 và Input Output Control by Identifier 0x2F. Các dịch vụ phức tạp khác như Security Access, nạp lại bộ nhớ Flash hay Diagnostic Session Control sẽ được dành cho hướng phát triển tương lai.

1.5 Bố cục của khóa luận

Nội dung khóa luận được cấu trúc thành 5 chương chính:

- Chương 1: Giới thiệu đề tài. Trình bày bối cảnh, lý do, mục tiêu, phạm vi nghiên cứu và giới thiệu cấu trúc của khóa luận.
- Chương 2: Cơ sở lý thuyết. Cung cấp các kiến thức nền tảng cần thiết về mạng truyền thông CAN, Automotive Ethernet, hệ điều hành FreeRTOS, cùng các tiêu chuẩn chẩn đoán DoIP và UDS.

- Chương 3: Thiết kế và hiện thực hệ thống. Phân tích chi tiết quá trình thiết kế sơ đồ phần cứng, cấu trúc phân tầng phần mềm, thuật toán xử lý đa tác vụ trên Firmware và kiến trúc của phần mềm giao diện Tester.
- Chương 4: Kết quả thực nghiệm và đánh giá. Trình bày hình ảnh mô hình thực tế, quá trình kiểm thử định tuyến, đánh giá các bản log gói tin mạng bằng Wireshark/CAN Analyzer và phân tích kết quả chẩn đoán.
- Chương 5: Kết luận và Hướng phát triển. Tổng kết những kết quả đã đạt được, đánh giá những ưu khuyết điểm của hệ thống và đề xuất các hướng nâng cấp, phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

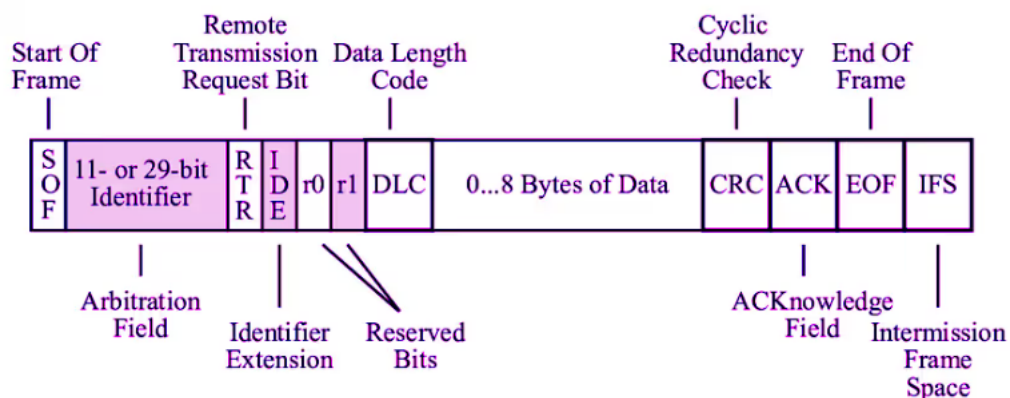
2.1 Mạng Controller Area Network (CAN)

Mạng CAN là một chuẩn mạng bus giao tiếp nối tiếp, đa chủ, hỗ trợ truyền thông thời gian thực với độ tin cậy cao trong môi trường công nghiệp và ô tô [8].

2.1.1 Cấu trúc chung của khung truyền dữ liệu mạng CAN

Về mặt lý thuyết, giao thức CAN định nghĩa cấu trúc khung truyền cho cả hai chuẩn: cơ bản (CAN 2.0A) và mở rộng (CAN 2.0B). Dựa trên đặc tả giao thức, một khung dữ liệu CAN gồm các phân đoạn cốt lõi sau:

- Start of Frame (SOF): Gồm 1 bit mức trệ (0) để đồng bộ hóa trạng thái bus giữa các nút trước khi truyền.
- Arbitration Field: Chứa ID định danh (11-bit với chuẩn 2.0A hoặc 29-bit với 2.0B) quyết định độ ưu tiên thông điệp (ID càng nhỏ ưu tiên càng cao), và bit RTR quy định loại khung (0: Khung dữ liệu, 1: Khung yêu cầu từ xa). Trong khóa luận này, hệ thống Gateway sử dụng chuẩn 11-bit và bit RTR bằng 0 để truyền dữ liệu thực.
- Control Field: Gồm bit IDE (giá trị 0 cho 11-bit, 1 cho 29-bit) để nhận diện chuẩn, 1 bit dự phòng r0, và 4 bit DLC chỉ định linh hoạt kích thước dữ liệu đi kèm (0 đến 8 bytes).
- Data Field: Chứa tải trọng ứng dụng (0 đến 8 bytes). Đây là trường lỗi do lập trình viên trực tiếp đóng gói và giải nén qua bộ đệm.
- Cyclic Redundancy Check (CRC): Mã kiểm tra 15-bit giúp các nút nhận phát hiện sai lệch dữ liệu do nhiễu vật lý.
- Acknowledge Field (ACK): Gồm 1 bit ACK và 1 bit phân tách. Nếu nút nhận thành công gói tin mà không phát hiện lỗi CRC, nó sẽ kéo bus xuống mức trệ để phản hồi xác nhận.
- End of Frame (EOF): Gồm 7 bit liên tiếp ở mức lặn (1) để đánh dấu sự kết thúc của một khung tin.



Hình 2.1: Cấu trúc chung của khung truyền dữ liệu mạng CAN

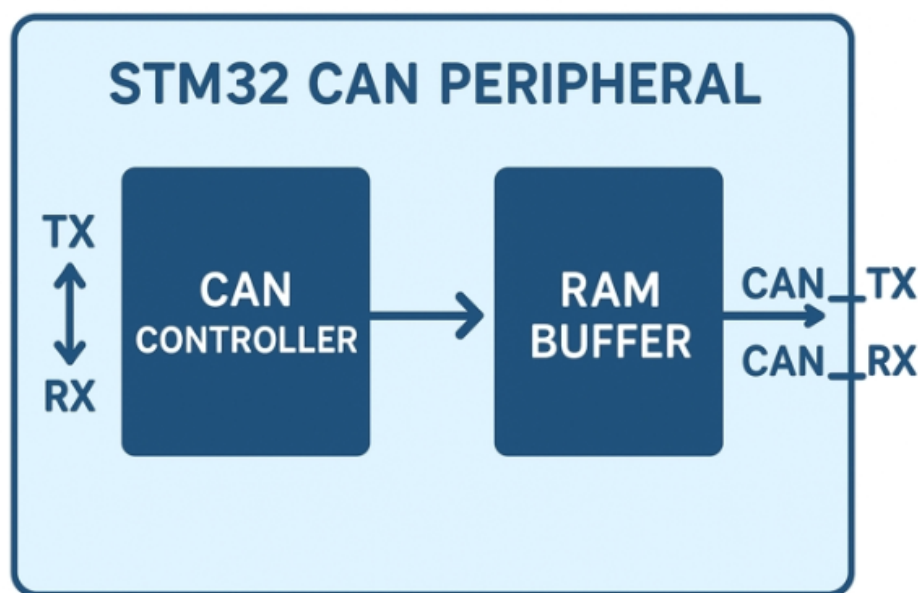
Điểm ưu việt khi triển khai trên các dòng vi điều khiển lõi ARM Cortex-M [3], điển hình là STM32, nằm ở chỗ toàn bộ quá trình tính toán mã CRC khắt khe cũng như việc phát/nhận bit ACK đều được khối phần cứng chuyên dụng tự động xử lý ở tầng vật lý. Điều này giúp giải phóng hoàn toàn gánh nặng thuật toán cho CPU, tạo tiền đề tối ưu cho các tác vụ định tuyến dữ liệu với độ trễ thấp ở các tầng trên.

2.1.2 Kiến trúc bộ điều khiển CAN trên vi điều khiển STM32

Bên cạnh các đặc tả lý thuyết chung, việc giao tiếp mạng CAN trong khóa luận được thực thi thông qua ngoại vi CAN tích hợp trên vi điều khiển STM32. Ngoại vi này được thiết kế để tự động quản lý toàn bộ các cơ chế giao thức mức thấp bao gồm: đóng gói khung truyền, phân xử đường truyền và xử lý lỗi [9].

Như được minh họa trong Hình 2.2, kiến trúc lõi của ngoại vi CAN trên STM32 bao gồm hai thành phần chính cùng các tính năng hỗ trợ trọng tâm cho việc xây dựng Gateway:

- Bộ điều khiển CAN: Trực tiếp quản lý trạng thái bus vật lý thông qua hai chân tín hiệu CAN_TX và CAN_RX. Nó hỗ trợ toàn diện cả định danh 11-bit và 29-bit.
- Cơ chế lọc tin nhắn phần cứng: Đây là tính năng cốt lõi để xây dựng Gateway. STM32 cung cấp các bảng lọc tích hợp sẵn, cho phép chỉ nhận các thông điệp có ID hợp lệ. Việc này giúp loại bỏ hoàn toàn các gói tin rác ngay từ tầng vật lý, giảm thiểu tối đa gánh nặng xử lý cho CPU và tăng hiệu suất định tuyến.
- Quản lý dữ liệu qua Ngắt và RAM Buffer: Dữ liệu sau khi qua bộ lọc sẽ được đẩy vào bộ đệm SRAM (Rx FIFOs). Sự kết hợp với cơ chế ngắt cho phép hệ thống FreeRTOS xử lý thông điệp cực kỳ nhanh chóng và hiệu quả ngay khi có gói tin mới được nhận vào bộ đệm.



Hình 2.2: Sơ đồ khối chức năng của ngoại vi điều khiển CAN trên vi điều khiển STM32

2.1.3 Cơ chế định tuyến chẩn đoán với Standard ID

Khác với chuẩn 29-bit có thể nhúng trực tiếp địa chỉ nguồn và đích vào ID, với chuẩn 11-bit, hệ thống chẩn đoán UDS quy định các cặp ID cố định để giao tiếp. Ví dụ, thiết bị kiểm tra sử dụng ID 0x7DF để gửi yêu cầu chức năng tới toàn mạng, hoặc 0x7E0 để gửi yêu cầu vật lý đích danh tới vi điều khiển đích. ECU sau đó sẽ phản hồi lại bằng ID 0x7E8.

Trong khóa luận này, bộ lọc phần cứng của các vi điều khiển được cấu hình ở chế độ List Mode 16-bit để tối ưu hóa hiệu suất tài nguyên. Khác với Mask Mode (lọc theo dải), List Mode thiết lập một danh sách định danh chính xác, từ chối mọi ID lạ. Cụ thể, Gateway được cấu hình để chỉ cho phép ID phản hồi chẩn đoán (0x7E8) và ID dữ liệu cảm biến (0x250) đi qua Rx FIFO. Tương tự, Node B (ECU) cũng sử dụng cơ chế này để chỉ nhận các yêu cầu chẩn đoán (0x7E0) và lệnh điều khiển (0x450). Thiết kế này giúp khóa chặt hệ thống định tuyến, ngăn chặn hoàn toàn các khung truyền không hợp lệ tiêu tốn tài nguyên xử lý của bus CAN.

2.2 Giao thức chẩn đoán hợp nhất UDS (ISO 14229)

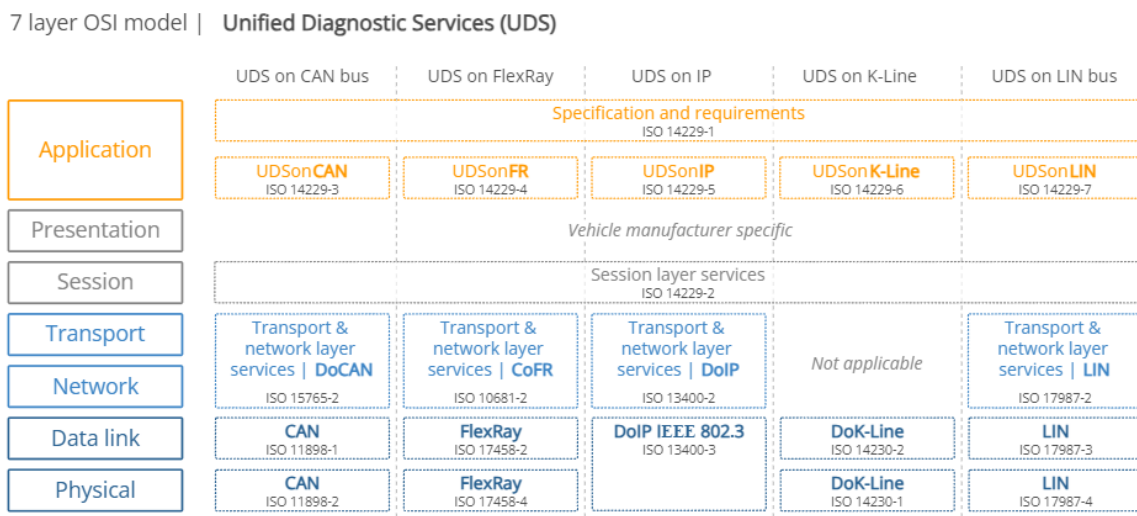
Giao thức UDS là tiêu chuẩn toàn cầu quy định các dịch vụ chẩn đoán nâng cao dành cho các bộ điều khiển điện tử (ECU) trên ô tô, được chuẩn hóa theo tiêu chuẩn quốc tế ISO 14229-1 [1]. Giao thức này hiện đang được áp dụng rộng rãi bởi hầu hết các nhà sản xuất thiết bị gốc (OEM).

2.2.1 Ứng dụng thực tiễn và Kiến trúc tham chiếu OSI

Trong thực tế, UDS mở ra khả năng can thiệp sâu vào hệ thống xe với các nhóm ứng dụng cốt lõi như:

- Đọc và xóa các mã lỗi chẩn đoán (DTC) để khắc phục sự cố.
- Trích xuất các tham số dữ liệu thời gian thực (ví dụ: khoảng cách cảm biến, nhiệt độ, số VIN).
- Khởi tạo các phiên chẩn đoán đặc biệt để kiểm thử các tính năng an toàn.
- Thay đổi hành vi của ECU thông qua các lệnh thiết lập lại (ECU Reset), nạp lại phần mềm (Flashing) và cấu hình thông số.

Xét trên mô hình tham chiếu OSI 7 tầng, UDS là một giao thức bậc cao. Nó hoạt động độc lập với môi trường vật lý bên dưới bằng cách định nghĩa các quy chuẩn chủ yếu ở Tầng Ứng dụng (Lớp 7) và quản lý trạng thái tại Tầng Phiên (Lớp 5).



Hình 2.3: Ảnh xạ kiến trúc giao thức UDS, DoCAN và DoIP trên mô hình tham chiếu OSI

Như được minh họa trong Hình 2.3, để gói tin UDS có thể truyền tải trên hệ thống mạng, nó phải dựa vào các giao thức Tầng Vận chuyển và Tầng Mạng (Lớp 3 và Lớp 4). Khi truyền trên mạng CAN, hệ thống sử dụng tiêu chuẩn DoCAN (**ISO 15765-2** / ISO-TP). Khi truyền trên nền tảng mạng Automotive Ethernet [5], hệ thống tuân theo tiêu chuẩn DoIP (**ISO 13400-2**). Sự phân tách rõ ràng giữa lớp ứng dụng và lớp truyền tải này chính là tiền đề lý thuyết then chốt để thiết kế và hiện thực hóa hệ thống Automotive Gateway đa giao thức trong khóa luận.

2.2.2 Kiến trúc thông điệp Yêu cầu và Phản hồi UDS

Mọi tiến trình giao tiếp UDS đều tuân thủ nghiêm ngặt cấu trúc Client-Server. Trong đó, Client đóng vai trò là thiết bị kiểm tra phát đi thông điệp yêu cầu, và Server là các bộ điều khiển điện tử hoặc hệ thống Gateway tiếp nhận xử lý. Cấu trúc của một gói tin yêu cầu cơ bản luôn bắt đầu bằng một byte mã dịch vụ (SID), theo sau có thể là các byte chứa hàm phụ hoặc tham số định danh dữ liệu tùy thuộc vào loại dịch vụ.

Quá trình truyền nhận thông điệp hỗ trợ hai phương thức định tuyến chính: định tuyến vật lý (giao tiếp điểm-điểm giữa Client và một Server duy nhất) dùng cho các tác vụ chuyên biệt, và định tuyến chức năng cho phép gửi yêu cầu đến nhiều Server cùng lúc. Sau khi tiếp nhận và giải mã thông điệp từ Client, Server sẽ trả về một trong hai kịch bản phản hồi sau:

1. Phản hồi tích cực: Gửi về khi yêu cầu hợp lệ và được thực thi thành công. Mã phản hồi được định dạng theo công thức: $\text{Response SID} = \text{Request SID} + 0x40$.
2. Phản hồi tiêu cực: Gửi về khi tiến trình thất bại hoặc không thỏa mãn điều kiện thực thi. Cấu trúc gói tin luôn bắt đầu bằng byte mặc định 0x7F, theo sau là byte SID bị từ chối và mã lỗi tiêu cực (NRC). Một số mã NRC phổ biến được hệ thống Gateway xử lý được trình bày chi tiết tại Bảng 2.1.

Bảng 2.1: Bảng tra cứu mã lỗi tiêu cực (NRC) phổ biến

Mã NRC	Tên mã lỗi kỹ thuật	Ý nghĩa ngữ cảnh xuất hiện
0x11	Service Not Supported	Mã dịch vụ SID không được Server hỗ trợ.
0x12	Sub-function Not Supported	Mã <i>Sub-function</i> yêu cầu không hợp lệ.
0x13	Incorrect Message Length Or Format	Chiều dài gói tin UDS sai lệch so với đặc tả.
0x22	Conditions Not Correct	Điều kiện ngoại vi của ECU chưa thỏa mãn để thực thi.
0x33	Security Access Denied	Dịch vụ bị khóa do chưa vượt qua Security Access.

2.2.3 Phân tích cấu trúc các dịch vụ UDS trong phạm vi nghiên cứu

Để hiện thực hóa mô hình Gateway, các dịch vụ sau được phân tích chi tiết về mặt cấu trúc byte và triển khai thực tế:

- ECU Reset (0x11): Điều khiển khởi động lại hệ thống ECU với các chế độ như Hard Reset (0x01) ngắt nguồn vật lý hoặc Key Off On Reset (0x02). Đối với kiến trúc Gateway, dịch vụ này đòi hỏi khả năng định tuyến bản tin cực kỳ chính xác để đảm bảo chỉ ECU mục tiêu trên mạng CAN con được khởi động lại mà không làm gián đoạn toàn bộ hệ thống.
- Read Data By Identifier (0x22): Dịch vụ cốt lõi để truy xuất dữ liệu từ ECU. Client cần gửi kèm mã định danh dữ liệu (DID) 2-byte. Trong khóa luận, dịch vụ này được tối ưu hóa bằng cơ chế Caching tại Gateway để trích xuất dữ liệu cảm biến thời gian thực với độ trễ thấp, đồng thời hỗ trợ gom nhóm truy vấn nhiều DID trong cùng một khung tin nhằm tiết kiệm băng thông mạng.
- Input Output Control By Identifier (0x2F): Đây là dịch vụ có độ phức tạp cao, cho phép can thiệp trực tiếp vào cơ cấu chấp hành. Nó yêu cầu truyền mã DID (trong khóa luận sử dụng DID 0x0102 đại diện cho hệ thống cảnh báo) kèm theo mã điều khiển *InputOutputControlParameter* (0x00: Trả quyền kiểm soát tự động cho ECU, 0x03: Kiểm soát giá trị tức thời từ Client). Đặc biệt ở chế độ 0x03, khung truyền yêu cầu thêm một byte *ControlState* mang giá trị từ 0 đến 3 để thiết lập các mức độ cảnh báo. Việc triển khai dịch vụ này đặt ra yêu cầu khắt khe về tính toàn vẹn dữ liệu và cơ chế kiểm soát mã lỗi tiêu cực (NRC) nhằm tránh các tác động vật lý không mong muốn lên xe.

Việc lựa chọn và tích hợp thành công ba nhóm dịch vụ cốt lõi nêu trên vào hệ thống Gateway không chỉ đáp ứng các yêu cầu chẩn đoán tiêu chuẩn quốc tế, mà còn tạo ra một nền tảng vững chắc để phát triển các tính năng điều khiển và giám sát xe đa luồng trong các chương tiếp theo của khóa luận.

2.3 Các giao thức tầng vận chuyển mạng ô tô

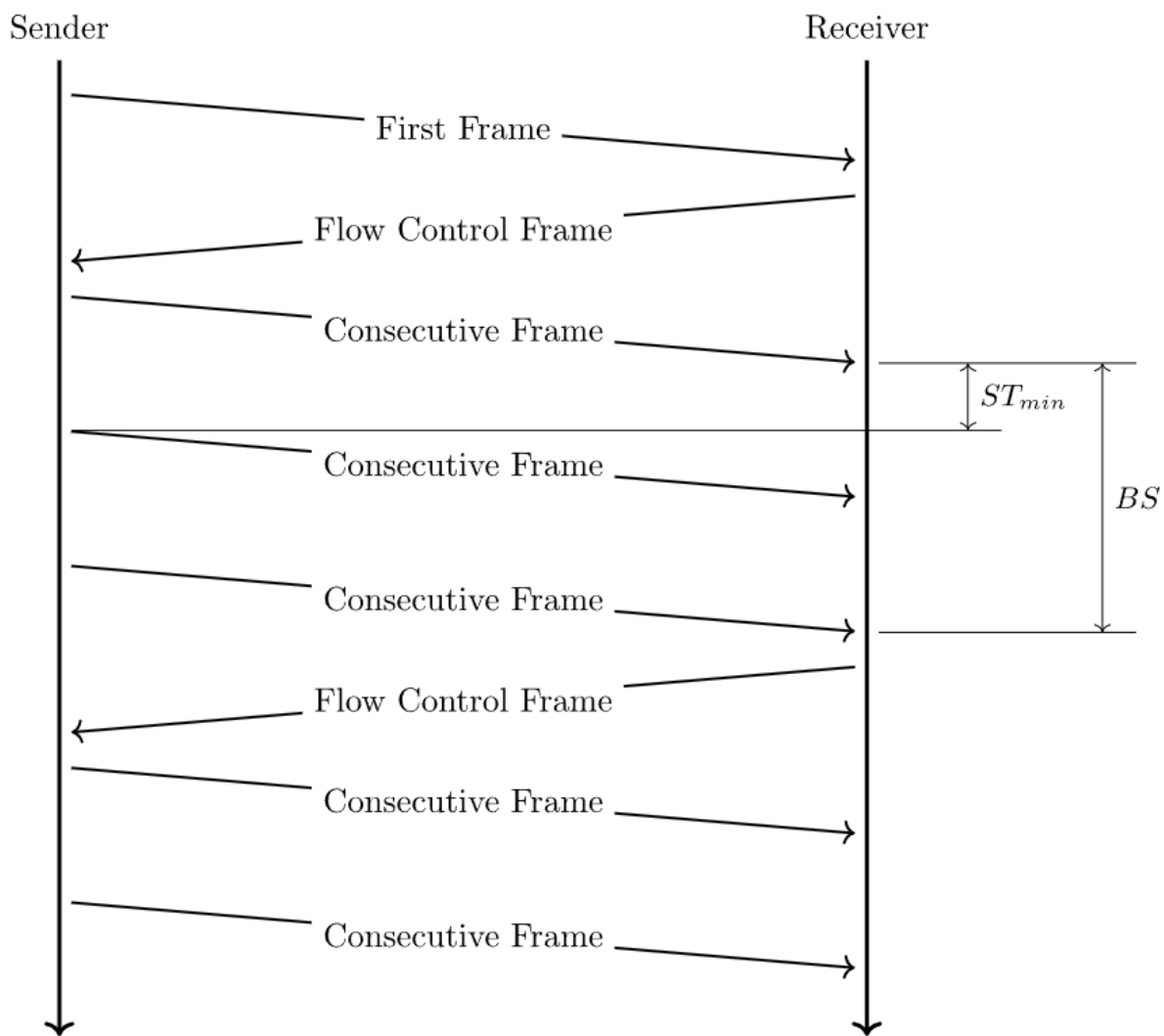
2.3.1 Giao thức vận chuyển qua mạng CAN (ISO-TP / ISO 15765-2)

Do mạng CAN truyền thống chỉ hỗ trợ kích thước tải trọng tối đa 8 byte cho mỗi khung truyền, việc trao đổi các thông điệp chẩn đoán UDS có kích thước lớn như nạp lại phần mềm hoặc đọc chuỗi dữ liệu dài là bất khả thi nếu không có cơ chế quản lý ở tầng vận chuyển. Tiêu chuẩn ISO 15765-2, hay còn gọi là ISO-TP, ra đời nhằm giải quyết triệt để bài toán này thông qua cơ chế phân mảnh và tái hợp dữ liệu.

Tiêu chuẩn này định nghĩa một quy trình truyền nhận chặt chẽ, sử dụng 4 bit cao của byte dữ liệu đầu tiên làm mã nhận dạng kiểu đơn vị dữ liệu giao thức (PDU Type). Dựa vào thông số này, hệ thống phân loại luồng giao tiếp thành 4 khung giao thức riêng biệt:

1. Single Frame (SF - Loại 0x0): Khung dữ liệu đơn, được sử dụng khi toàn bộ tải trọng UDS có thể nằm gọn trong giới hạn 7 bytes. Trong cấu trúc này, Byte 0 chứa mã kiểu khung và thông tin về chiều dài dữ liệu thực tế.
2. First Frame (FF - Loại 0x1): Khung khởi đầu quá trình phân mảnh, phát đi khi thông điệp dài hơn 7 bytes. Hai byte đầu tiên chứa mã kiểu khung và tổng chiều dài toàn bộ gói tin được biểu diễn dưới dạng 12-bit, cho phép truyền tải khối dữ liệu lên đến 4095 bytes.
3. Consecutive Frame (CF - Loại 0x2): Các khung mang dữ liệu phân mảnh bám sát theo sau khung khởi đầu. Byte 0 chứa mã kiểu khung và mã số đếm tuần tự thay đổi từ 1 đến 15 rồi lặp lại về 0, giúp bên nhận kiểm tra tính toàn vẹn và đúng thứ tự của luồng gói tin.
4. Flow Control (FC - Loại 0x3): Khung điều khiển luồng do bên nhận phát ngược lại nhằm điều tiết tốc độ truyền của bên phát. Khung này mang các tham số quan trọng bao gồm trạng thái luồng ở Byte 0, kích thước khối quy định số lượng khung dữ liệu liên tiếp được phép gửi ở Byte 1, và thời gian giãn cách tối thiểu giữa hai khung liên tiếp (STmin) ở Byte 2.

Nhờ sự phối hợp đồng bộ giữa các khung dữ liệu và khung điều khiển luồng này, quá trình truyền tải dữ liệu chẩn đoán kích thước lớn qua bus CAN diễn ra an toàn, giảm thiểu tối đa hiện tượng tràn bộ đệm đối với các bộ điều khiển có tài nguyên hạn hẹp.



Hình 2.4: Cấu trúc ánh xạ và lưu đồ phân mảnh gói tin theo tiêu chuẩn ISO-TP

2.3.2 Giao thức chẩn đoán qua nền tảng Internet Protocol (DoIP / ISO 13400)

Tiêu chuẩn ISO 13400 (DoIP) dịch chuyển các lệnh chẩn đoán từ môi trường bus nối tiếp sang môi trường mạng Ethernet bằng cách đóng gói thông điệp chẩn đoán vào các gói tin TCP/UDP [2]. Một gói tin DoIP tiêu chuẩn bắt buộc phải có cấu trúc Header cố định kích thước 8 bytes trước khi đến phần tải trọng UDS.

Bảng 2.2: Đặc tả cấu trúc cấu thành 8 byte DoIP Header chuẩn ISO 13400

Phân đoạn Byte	Kích thước	Tên trường dữ liệu	Giá trị định cấu hình
Byte 0	1 byte	Protocol Version	0x02 (Bản chuẩn ISO 13400-2:2012)
Byte 1	1 byte	Inverse Version	0xFD (Mã đảo bit logic của Byte 0)
Byte 2 - 3	2 bytes	Payload Type	Định nghĩa kiểu dữ liệu (Ví dụ: 0x8001)
Byte 4 - 7	4 bytes	Payload Length	Chiều dài phần dữ liệu UDS phía sau (bytes)

Trường *Payload Type* quyết định cách thức Gateway xử lý gói tin. Các mã định danh *Payload Type* cốt lõi bao gồm:

- 0x0001 / 0x0004: Tin nhắn yêu cầu nhận diện và phản hồi thông tin định danh phương tiện.
- 0x0005 / 0x0006: Tin nhắn yêu cầu và phản hồi kích hoạt định tuyến thiết lập kết nối logic giữa Client và Gateway.
- 0x8001 / 0x8002 / 0x8003: Tin nhắn chẩn đoán chính thức có tải trọng chứa các hàm dịch vụ UDS.

Dựa trên nền tảng của kiến trúc DoIP (ISO 13400), hệ thống Gateway được thiết kế để tuân thủ quy chuẩn đóng gói bản tin chẩn đoán khi giao tiếp luồng dữ liệu giữa mạng Ethernet và mạng CAN. Cụ thể, hệ thống thực hiện trích xuất và đóng gói dữ liệu với đầy đủ cấu trúc Header 8 bytes (bao gồm *Protocol Version* 0x02, *Inverse Version* 0xFD, và *Payload Type* 0x8001).

Khi nhận lệnh từ giao diện điều khiển qua nền tảng IP, Gateway sẽ bóc tách lớp Header DoIP để lấy tải trọng UDS và định tuyến xuống các node phần cứng qua bus CAN. Ở chiều ngược lại, dữ liệu phản hồi từ mạng CAN được Gateway bọc lại một lớp DoIP Header hoàn chỉnh trước khi truyền tải qua UDP về trình quản lý. Việc hiện thực hóa thành công cấu trúc tiêu chuẩn này không chỉ đảm bảo tính đồng bộ với các hệ thống chẩn đoán ô tô hiện đại mà còn là cơ sở vững chắc để kiểm chứng, phân tích tính chính xác của luồng định tuyến thông qua phần mềm Wireshark.

2.3.3 Kiến trúc Phần mềm nhúng đa nhiệm FreeRTOS và LwIP

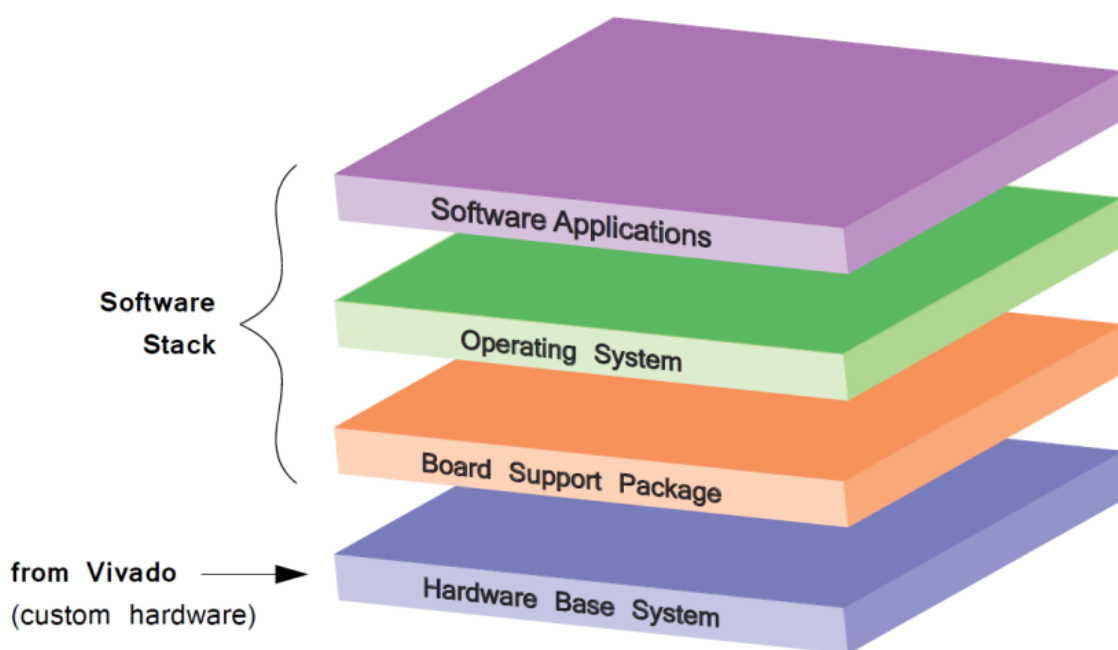
Hệ thống Gateway đóng vai trò đồng bộ hóa hai mạng có đặc thù truyền tải khác biệt: mạng CAN yêu cầu độ trễ xử lý ngắn cực thấp, trong khi mạng Ethernet có băng thông lớn với luồng dữ liệu UDP/IP tốc độ cao. Kiến trúc vòng lặp vô hạn truyền thống không thể đáp ứng và dễ gây mất gói tin. Nếu xử lý tuần tự, vi điều khiển có thể bị nghẽn khi phân tích gói mạng, dẫn đến tràn bộ đệm và bỏ lỡ các khung truyền CAN khẩn cấp.

Do đó, hệ thống cần một hệ điều hành thời gian thực (RTOS) cung cấp khả năng đa nhiệm với thời gian đáp ứng mang tính xác định. Khóa luận triển khai kiến trúc đa nhiệm kết hợp nền tảng FreeRTOS [4] và ngăn xếp mạng mã nguồn mở LwIP. Giải pháp này giúp phân tách độc lập luồng thu phát thành các tác vụ riêng biệt, đồng thời quản lý hiệu quả tài nguyên bộ nhớ khi định tuyến liên tục gói tin DoIP/UDS qua giao thức UDP.

2.3.4 Kiến trúc phân tầng phần mềm

Theo tiêu chuẩn thiết kế hệ thống nhúng hiện đại, phần mềm Gateway được chia thành 4 phân tầng biệt lập để tăng tính linh hoạt và dễ dàng bảo trì:

- Hardware Base System: Nền tảng vi điều khiển vật lý STM32 và các linh kiện điện tử ngoại vi đi kèm.
- Board Support Package (BSP): Lớp thư viện trừu tượng phần cứng, tiêu biểu là thư viện HAL của hệ sinh thái STM32. Lớp này cung cấp các giao diện lập trình chuẩn hóa (API) để khởi tạo và cấu hình trực tiếp các thành ghi mà không yêu cầu can thiệp quá sâu vào mã máy mức thấp.
- Operating System (OS): Lõi hệ điều hành FreeRTOS [4] và ngăn xếp mạng LwIP đóng vai trò điều phối luồng thực thi, cấp phát và quản lý tài nguyên bộ nhớ.
- Software Applications: Lớp ứng dụng trên cùng chứa các luồng tác vụ định tuyến CAN, xử lý UDP và các thuật toán hệ thống.

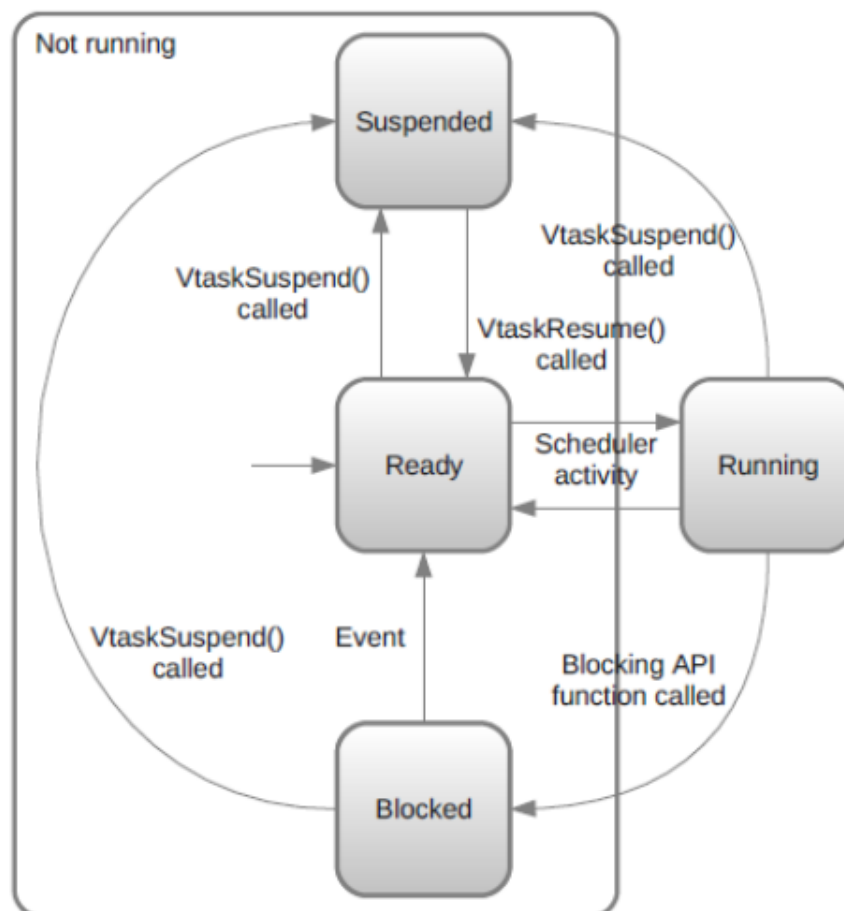


Hình 2.5: Kiến trúc phân tầng phần mềm hệ thống nhúng với RTOS

2.3.5 Vòng đời và cơ chế chuyển đổi trạng thái của luồng tác vụ

Hệ điều hành FreeRTOS điều khiển tài nguyên CPU thông qua bộ lập lịch ưu tiên độc quyền. Tại một thời điểm, vi điều khiển lõi đơn chỉ có thể thực thi một tác vụ duy nhất. Do đó, vòng đời của một tác vụ liên tục luân chuyển qua 4 trạng thái cốt lõi:

1. Running: Tác vụ đang chiếm dụng CPU để chạy các tập lệnh.
2. Ready: Tác vụ đã đủ điều kiện để chạy nhưng phải nhường CPU cho một luồng khác có mức ưu tiên cao hơn.
3. Blocked: Tác vụ đang tạm dừng để chờ một sự kiện xảy ra, ví dụ như chờ một khoảng thời gian trễ, chờ cờ hiệu, hoặc chờ tín hiệu từ các hàng đợi liên luồng (như CAN to Ethernet Queue được triển khai trong hệ thống).
4. Suspended: Tác vụ bị vô hiệu hóa hoàn toàn khỏi bộ lập lịch cho đến khi được gọi hàm đánh thức một cách tường minh.



Hình 2.6: Sơ đồ chuyển đổi trạng thái của một luồng tác vụ trong FreeRTOS

2.3.6 Ngăn xếp mạng rút gọn LwIP trong hệ thống nhúng

Ngăn xếp mạng LwIP là một giải pháp cung cấp đầy đủ các giao thức họ TCP/IP (bao gồm UDP, TCP, IPv4) cùng giao diện Socket API tương tự chuẩn Berkeley. Điểm ưu việt nhất của LwIP là kiến trúc tối ưu hóa tài nguyên, chỉ yêu cầu khoảng 40 KB bộ nhớ Flash và 8 KB bộ nhớ RAM.

Khóa luận sử dụng mô hình RAW API dựa trên cơ chế Callback của LwIP, kết hợp chặt chẽ với bộ điều khiển lớp MAC Ethernet tích hợp sẵn trong vi điều khiển. Cơ chế này giúp bỏ qua độ trễ luân chuyển tác vụ của hệ điều hành, cho phép tiếp nhận gói tin DoIP từ phần mềm máy tính truyền xuống và lập tức định tuyến bản tin UDS vào mạng CAN với độ trễ thấp nhất.

CHƯƠNG 3. THIẾT KẾ VÀ HIỆN THỰC HỆ THỐNG

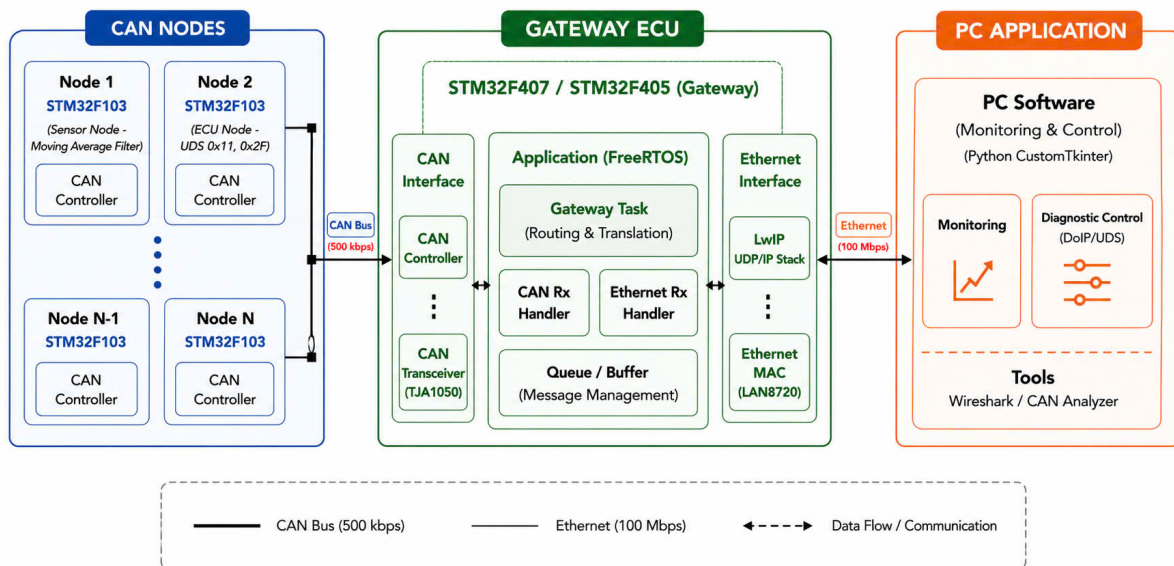
3.1 Tổng quan kiến trúc hệ thống

Dựa trên các yêu cầu về chẩn đoán ô tô và định tuyến bản tin, hệ thống Gateway CAN-Ethernet được thiết kế với ba phân hệ chính, hoạt động phối hợp để đảm bảo tính toàn vẹn và tốc độ truyền tải dữ liệu. Cấu trúc tổng thể được minh họa trong Hình 3.1.

Phân hệ đầu tiên là khối chấp hành và cảm biến. Khối này đóng vai trò như một thiết bị điều khiển điện tử (ECU) thực tế trên xe ô tô, giao tiếp trực tiếp với hệ thống thông qua mạng CAN và hoạt động như một máy chủ chẩn đoán (UDS Server). Được tích hợp cảm biến siêu âm, khối này đảm nhiệm việc đo đạc khoảng cách thời gian thực. Dữ liệu thô từ cảm biến sẽ được đưa qua bộ lọc nhiễu số trung bình trượt số mũ trực tiếp trên phần cứng trước khi phát lên bus dữ liệu. Đồng thời, node mạng luôn duy trì trạng thái chờ nhận các mã lệnh UDS được truyền xuống nhằm thực thi các tác vụ như khởi động lại phần cứng hoặc điều khiển cơ cấu chấp hành.

Phân hệ thứ hai, đồng thời là thành phần cốt lõi của toàn bộ mô hình, là khối trung tâm định tuyến (Gateway). Khối này được xây dựng trên nền tảng vi điều khiển STM32 kết hợp với hệ điều hành thời gian thực FreeRTOS, có nhiệm vụ giám sát dữ liệu song song trên hai cổng giao tiếp vật lý: cổng RJ45 cho mạng Ethernet và bộ thu phát cho mạng CAN. Quá trình định tuyến diễn ra theo hai chiều: khi nhận được gói tin DoIP từ máy tính, Gateway tiến hành bóc tách lớp Header, chuyển đổi thông điệp sang chuẩn khung truyền CAN 2.0A và đẩy xuống bus CAN; ngược lại, hệ thống tiếp nhận dữ liệu cảm biến từ bus CAN, đóng gói thành bản tin Ethernet và truyền ngược lên máy tính. Bên cạnh đó, Gateway được tích hợp cơ chế Caching để xử lý các yêu cầu truy xuất dữ liệu từ phần mềm quản lý, góp phần giảm độ trễ truyền dẫn của hệ thống.

Phân hệ cuối cùng là khối giám sát và điều khiển. Phân hệ này được triển khai trên nền tảng máy tính cá nhân dưới dạng một phần mềm ứng dụng phát triển bằng ngôn ngữ Python, sử dụng thư viện giao diện customtkinter [7]. Khối này đóng vai trò là một thiết bị kiểm tra (UDS Client), cung cấp giao diện trực quan cho phép người dùng giám sát biểu đồ dữ liệu, lưu trữ nhật ký hoạt động dưới định dạng CSV và chủ động phát đi các lệnh chẩn đoán UDS (điển hình như 0x11, 0x22, 0x2F). Các mã lệnh này được đóng gói sẵn trong phần tải trọng của giao thức DoIP và truyền tải qua môi trường mạng Ethernet theo chuẩn UDP/IPv4.



Hình 3.1: Sơ đồ khối tổng thể kiến trúc hệ thống Gateway CAN-Ethernet

3.2 Thiết kế phần cứng hệ thống

Hệ thống Gateway và các node mạng được thiết kế dựa trên nguyên tắc đảm bảo tính ổn định trong truyền thông, tối ưu hóa chi phí nhưng vẫn đáp ứng đầy đủ yêu cầu về băng thông cho các giao thức chẩn đoán. Cấu trúc phần cứng được chia thành hai khối chính: Khối trung tâm định tuyến (Gateway) và Khối chấp hành/cảm biến (Node).

3.2.1 Lựa chọn vi điều khiển trung tâm

Dựa trên yêu cầu xử lý đa tác vụ của hệ điều hành FreeRTOS và ngăn xếp mạng LwIP, nền tảng vi điều khiển kiến trúc ARM Cortex-M [3] được lựa chọn để làm lõi xử lý.

- Đối với khối Gateway: Vi điều khiển STM32F4 (lõi Cortex-M4) được sử dụng nhờ tích hợp sẵn bộ điều khiển MAC hỗ trợ chuẩn Fast Ethernet 10/100 Mbps. Đồng thời, dung lượng bộ đệm RAM nội bộ của dòng vi điều khiển này đủ lớn để lưu trữ, phân mảnh các gói tin UDP tải trọng cao từ giao thức DoIP và phục vụ cơ chế *Gateway Caching*.
- Đối với khối Node chẩn đoán: Vi điều khiển STM32F1 (lõi Cortex-M3) được triển khai do đáp ứng tốt các yêu cầu về ngoại vi CAN 2.0B và tốc độ xử lý tín hiệu ngắt từ cảm biến ở tầng vật lý, đồng thời giúp tối ưu hóa giá thành triển khai cho các nút mạng cấp thấp.

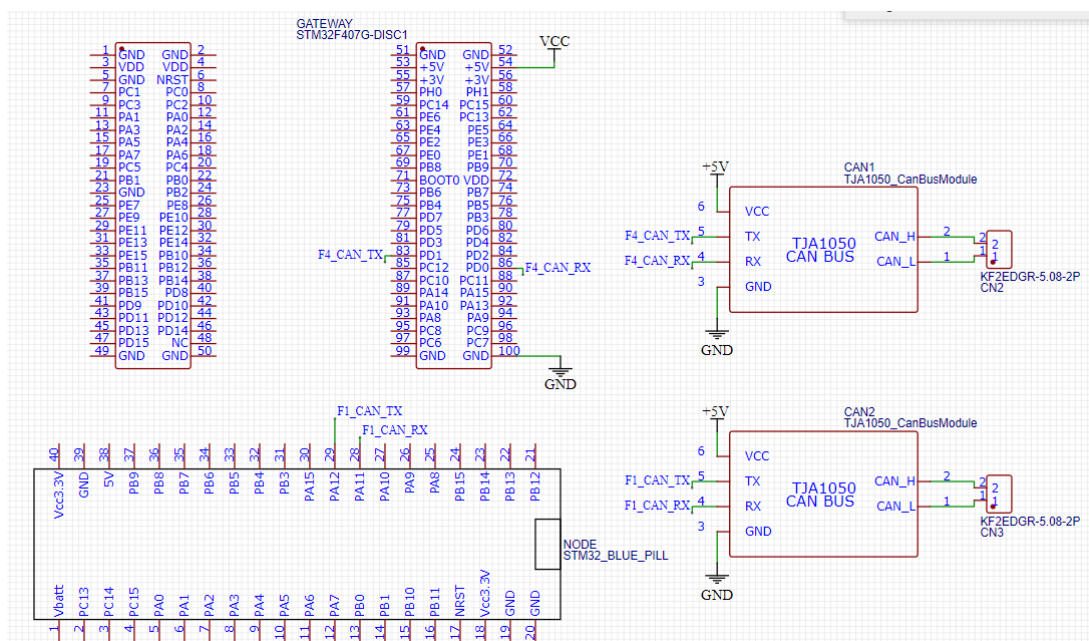
3.2.2 Thiết kế mạch giao tiếp mạng CAN

Đề vi điều khiển STM32 có thể giao tiếp trên bus CAN vật lý, tín hiệu logic (Rx/Tx) từ vi điều khiển cần được chuyển đổi thành tín hiệu vi sai (CAN_H và CAN_L). IC thu phát TJA1050 được lựa chọn làm bộ thu phát CAN nhờ khả năng chống nhiễu điện từ (EMI) ở mức cao và tương thích hoàn toàn với tiêu chuẩn ISO 11898.

Một điện trở đầu cuối 120 Ω được bố trí giữa hai đường tín hiệu vi sai để giảm thiểu hiện tượng dội sóng ở tần số cao. Quá trình ghép nối vật lý được mô tả chi tiết trong Bảng 3.1.

Bảng 3.1: Bảng sơ đồ kết nối phần cứng giữa vi điều khiển (F4/F1) và IC thu phát CAN TJA1050

Chân F4	Chân F1	Chân TJA1050	Chức năng	Mô tả
5V	5V	VCC	Power	Nguồn cung cấp 5V cho IC thu phát
GND	GND	GND	Ground	Nối đất chung toàn hệ thống
PD0	PA11	RXD	CAN_RX	Chân nhận dữ liệu CAN từ bus vật lý về vi điều khiển
PD1	PA12	TXD	CAN_TX	Chân phát dữ liệu CAN từ vi điều khiển ra bus vật lý



Hình 3.2: Sơ đồ nguyên lý mạch giao tiếp CAN sử dụng IC TJA1050

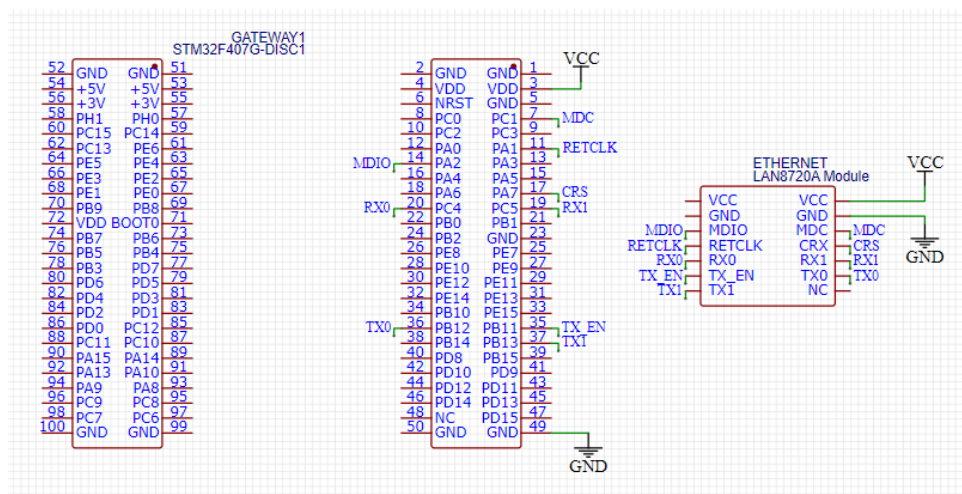
3.2.3 Thiết kế mạch giao tiếp Ethernet (PHY)

Vi điều khiển STM32F4 chỉ tích hợp bộ điều khiển MAC, do đó yêu cầu một IC tầng vật lý (PHY) bên ngoài để điều chế và truyền nhận tín hiệu lên cáp mạng đôi xoắn. IC LAN8720A được sử dụng để đảm nhận vai trò này.

Giao tiếp giữa MAC (STM32F4) và PHY (LAN8720A) được thực hiện thông qua giao diện rút gọn RMII. Chuẩn giao tiếp này giúp tối ưu hóa số lượng chân vi điều khiển so với chuẩn MII truyền thống. Tín hiệu xung nhịp đồng bộ 50 MHz được cấp cho cả mạch MAC và PHY để đảm bảo tính toàn vẹn của luồng dữ liệu mạng.

Bảng 3.2: Bảng sơ đồ kết nối giao diện RMII giữa STM32F4 và IC LAN8720A

Chân STM32F4	Chân LAN8720A	Chức năng	Mô tả
PC1	MDC	ETH_MDC	Xung nhịp quản lý dữ liệu
PA2	MDIO	ETH_MDIO	Kênh dữ liệu quản lý hai chiều
PA1	REF_CLK	ETH_REF_CLK	Xung nhịp tham chiếu đồng bộ 50 MHz
PA7	CRS_DV	ETH_CRS_DV	Tín hiệu báo dữ liệu nhận hợp lệ
PC4	RXD0	ETH_RXD0	Kênh nhận dữ liệu mạng số 0
PC5	RXD1	ETH_RXD1	Kênh nhận dữ liệu mạng số 1
PB11	TX_EN	ETH_TX_EN	Tín hiệu kích hoạt truyền dữ liệu
PB12	TXD0	ETH_TXD0	Kênh truyền dữ liệu mạng số 0
PB13	TXD1	ETH_TXD1	Kênh truyền dữ liệu mạng số 1



Hình 3.3: Sơ đồ nguyên lý giao tiếp RMII và cấu hình IC Ethernet LAN8720A

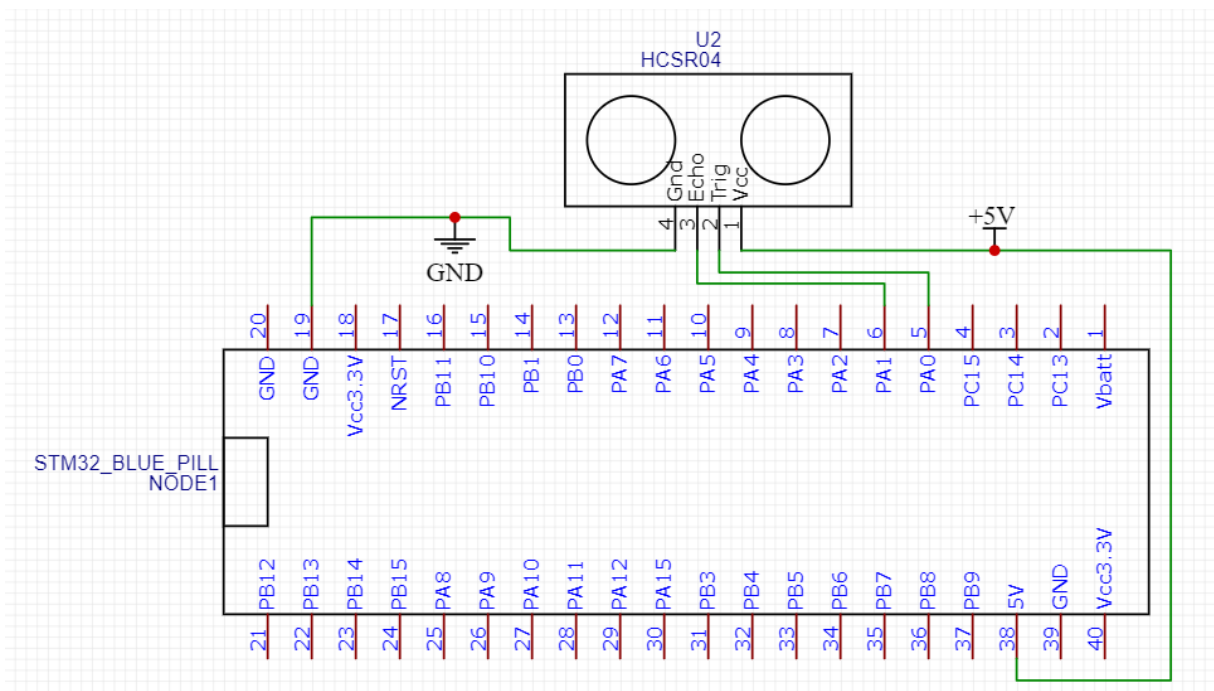
3.2.4 Thiết kế khối thu thập dữ liệu Node chẩn đoán

Tại thiết bị chấp hành (CAN Node), dữ liệu khoảng cách vật lý được thu thập liên tục thông qua module cảm biến siêu âm nhằm giả lập trạng thái hoạt động của phương tiện. Tín hiệu đo đạc được đưa vào chân GPIO của STM32F1 thông qua cấu hình tạo xung và ngắt ngoài dựa trên cơ chế xuất/nhập tín hiệu của cảm biến.

Nhằm hạn chế nhiễu đo lường, cấu trúc phần cứng được thiết kế kèm tụ lọc nguồn và các chân tín hiệu được định tuyến ngắn nhất có thể. Dữ liệu sau khi thu thập sẽ được đưa vào các thuật toán lọc số trước khi hệ điều hành đóng gói thành khung tin CAN.

Bảng 3.3: Bảng sơ đồ kết nối phần cứng giữa Node STM32F1 và module Cảm biến siêu âm

Chân Cảm biến	Chân STM32F1	Chức năng	Mô tả
VCC	5V	Power	Nguồn cung cấp điện áp hoạt động cho cảm biến
GND	GND	Ground	Nối đất chung
TRIG	PA0	Trigger / Output	Chân vi điều khiển xuất xung kích hoạt quá trình đo lường
ECHO	PA1	Echo / Ext. Int.	Chân vi điều khiển nhận xung phản hồi để tính toán thời gian



Hình 3.4: Sơ đồ nguyên lý kết nối giữa node A và cảm biến siêu âm HC-SR04

3.3 Thiết kế phần mềm và thuật toán điều khiển

Bên cạnh thiết kế phần cứng, hệ thống phần mềm đóng vai trò cốt lõi trong việc đảm bảo khả năng định tuyến dữ liệu theo thời gian thực giữa mạng CAN và giao thức DoIP.

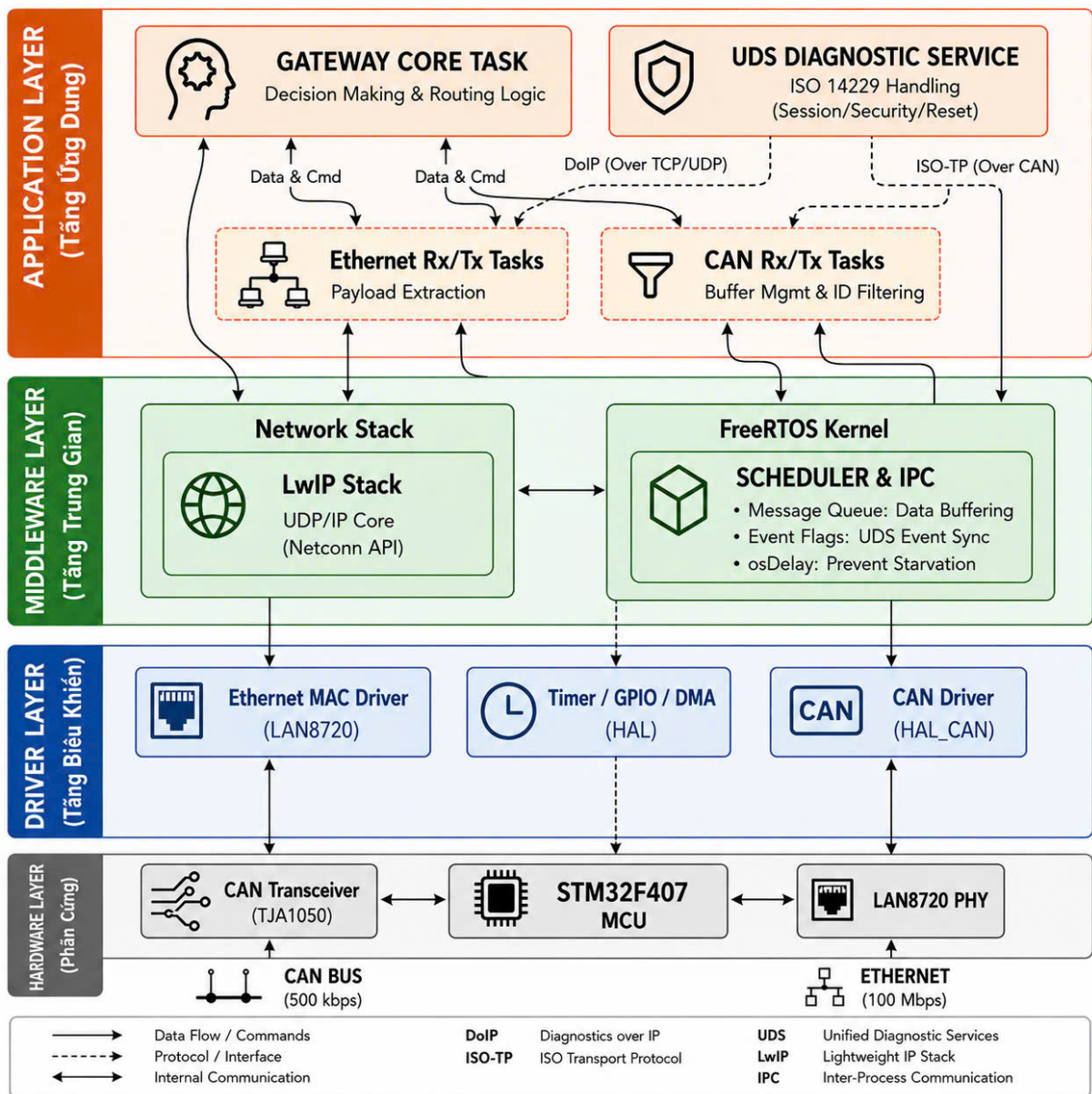
3.3.1 Kiến trúc phần mềm tổng thể

Để đảm bảo tính module hóa, dễ dàng bảo trì và mở rộng trong tương lai, kiến trúc phần mềm của hệ thống được thiết kế dựa trên mô hình phân lớp. Cấu trúc này giúp tách biệt logic điều khiển ứng dụng khỏi các phụ thuộc vật lý của phần cứng thông qua ba phân tầng chính.

Nằm ở vị trí dưới cùng là lớp trừu tượng phần cứng (MCAL), chịu trách nhiệm giao tiếp trực tiếp với vi điều khiển. Lớp này khai thác hệ sinh thái thư viện HAL do STMicroelectronics cung cấp nhằm trừu tượng hóa các thanh ghi phức tạp, đồng thời cung cấp hệ thống giao diện lập trình (API) chuẩn mực để khởi tạo và điều khiển các ngoại vi cốt lõi như bộ điều khiển CAN, khối MAC Ethernet, các chân GPIO và bộ chuyển đổi ADC.

Kế tiếp là lớp trung gian, đóng vai trò như một cầu nối cung cấp các dịch vụ hệ thống và nền tảng thực thi ổn định cho lớp ứng dụng. Cụ thể tại thiết bị Gateway trung tâm (STM32F4), lớp này tích hợp hệ điều hành thời gian thực FreeRTOS [6] để quản lý tài nguyên và điều phối đa tác vụ. Hoạt động song song với lõi RTOS là ngăn xếp mạng LwIP, đảm nhận nhiệm vụ xử lý toàn vẹn chu trình truyền nhận của các giao thức TCP/UDP/IPv4.

Phân tầng trên cùng là lớp ứng dụng, nơi chứa toàn bộ logic hoạt động cốt lõi của hệ thống. Tại khối Gateway, lớp này bao gồm các luồng tác vụ định tuyến, đảm nhiệm việc bóc tách và chuyển đổi qua lại giữa khung tin CAN và định dạng DoIP. Trong khi đó, tại thiết bị chấp hành (Node), lớp ứng dụng thực thi các thuật toán thu thập dữ liệu cảm biến, lọc nhiễu tín hiệu và đóng gói bản tin lên bus CAN. Việc tuân thủ nghiêm ngặt kiến trúc 3 lớp này đảm bảo hệ thống đạt tính cô lập cao; khi có sự thay đổi về nền tảng phần cứng, nhà phát triển chỉ cần cập nhật lại cấu hình tại lớp MCAL mà không làm xáo trộn các thuật toán định tuyến DoIP hay logic điều khiển ở lớp ứng dụng.



Hình 3.5: Sơ đồ kiến trúc phần mềm 3 lớp (MCAL, Middleware, Application) của hệ thống

3.3.2 Kiến trúc đa nhiệm với FreeRTOS

Hệ thống Gateway yêu cầu khả năng xử lý song song các tác vụ mạng (Ethernet/LwIP) và các ngắt phần cứng (CAN bus) với độ trễ thấp nhất. Do đó, hệ điều hành thời gian thực FreeRTOS được tích hợp để điều phối tài nguyên CPU. Cấu trúc đa nhiệm của hệ thống được thiết kế với các tác vụ và mức độ ưu tiên cụ thể như trong Bảng 3.4.

Bảng 3.4: Cấu hình các luồng tác vụ trong FreeRTOS

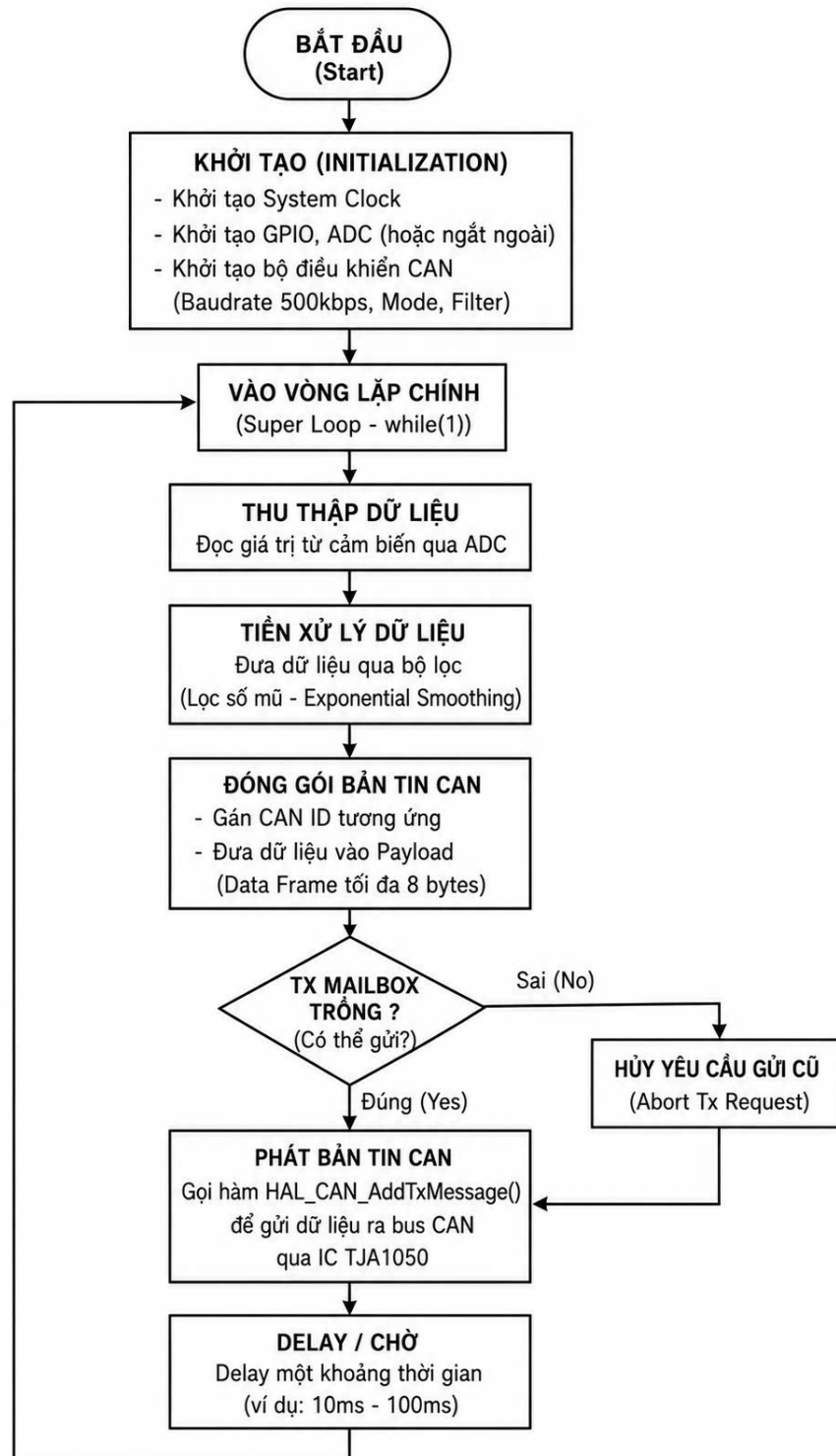
Tên tác vụ	Mức ưu tiên	Stack Size	Chức năng chính
CanRxTask	High	256 Words	Xử lý ngắt nhận dữ liệu từ bus CAN, đẩy khung truyền thô vào hàng đợi.
EthLinkTask	Above Normal	512 Words	Quản lý vòng đời kết nối Ethernet và ngăn xếp LwIP.
GatewayRoutingTask	Normal	512 Words	Xử lý logic cốt lõi: Bóc tách Header DoIP, phân tích mã dịch vụ UDS và tính tiền giao thức.
MonitorTask	Low	128 Words	Nháy LED Heartbeat, in log UART để gỡ lỗi và giám sát trạng thái tràn bộ nhớ.

Bên cạnh việc phân chia tác vụ, hệ thống sử dụng các cơ chế giao tiếp liên tiến trình (IPC) được cung cấp bởi FreeRTOS để đảm bảo tính đồng bộ và toàn vẹn dữ liệu:

- Hàng đợi thông điệp (osMessageQueue): Được cấu hình như một bộ đệm luân chuyển dữ liệu an toàn giữa các luồng. Cụ thể, CAN_Data_QueueHandle tiếp nhận dữ liệu cảm biến từ mạng CAN, trong khi Eth_Data_QueueHandle làm nhiệm vụ chuyển tiếp dữ liệu đã qua bộ lọc số từ tác vụ xử lý logic sang tác vụ Ethernet để đẩy lên giao diện giám sát trên máy tính theo thời gian thực.
- Cờ sự kiện (osEventFlags): Cờ sự kiện (UdsEventFlagsHandle) được áp dụng tại tác vụ xử lý logic để đồng bộ hóa cơ chế nhận phản hồi chẩn đoán UDS. Hệ thống sẽ chờ cờ EVT_UDS_RX_BIT được kích hoạt từ ngắt, cho phép vi điều khiển đánh giá chính xác thời điểm có khung truyền UDS hợp lệ mà không lãng phí chu kỳ máy.
- Hàm nhường quyền CPU (osDelay): Các luồng tác vụ đều được thiết kế kết hợp với hàm nhường quyền điều khiển (osDelay), đảm bảo hệ điều hành luôn có đủ định mức thời gian để quét và thực thi các tác vụ có mức ưu tiên thấp hơn (như MonitorTask), ngăn ngừa triệt để tình trạng treo hệ thống do thiếu tài nguyên.

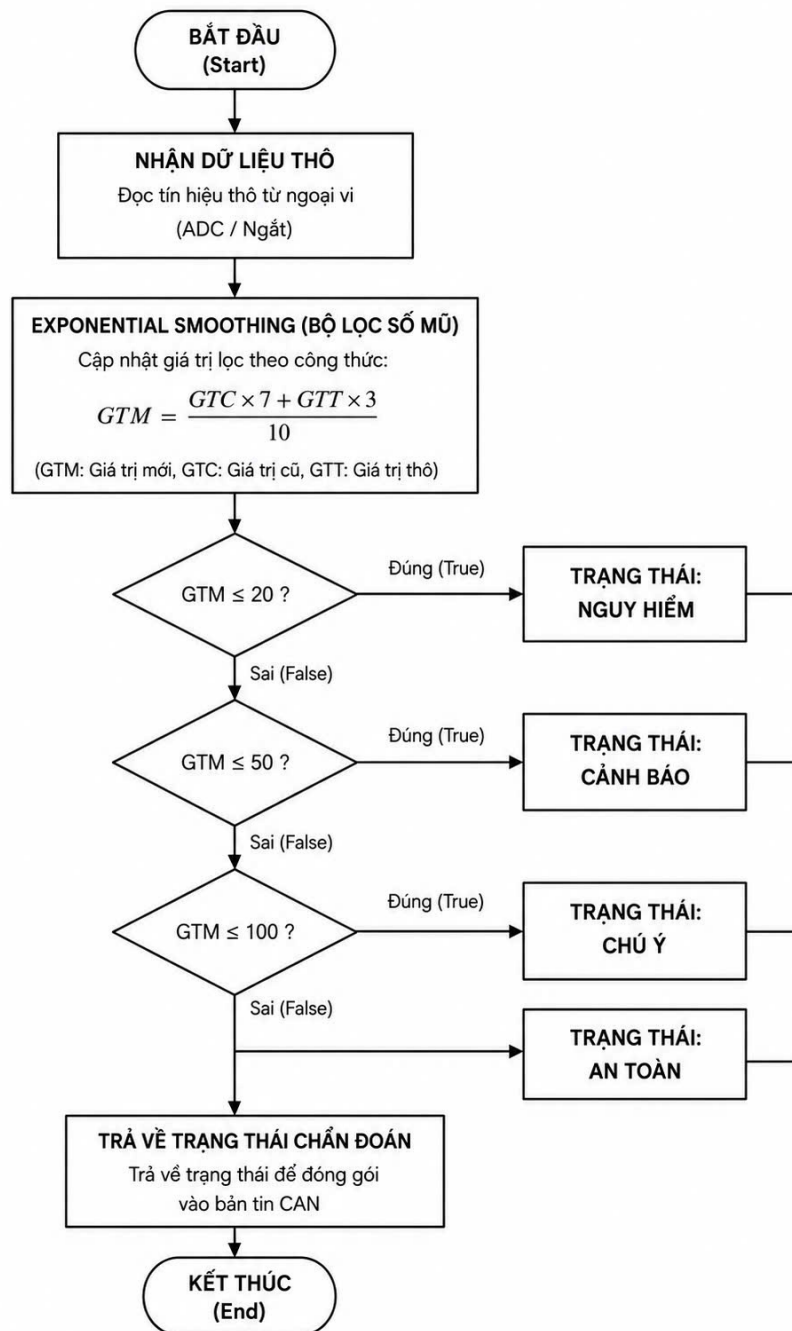
3.3.3 Thuật toán điều khiển node chẩn đoán (STM32F1)

Tại các node đầu cuối, luồng chương trình hoạt động theo mô hình vòng lặp vô hạn kết hợp với điều khiển ngắt để tối ưu hóa thời gian phản hồi. Toàn bộ quy trình hoạt động tổng thể của vi điều khiển được thể hiện trong Hình 3.6.



Hình 3.6: Lưu đồ thuật toán điều khiển chương trình chính tại node STM32F1

Giai đoạn thứ nhất là thu thập và tiền xử lý dữ liệu. Vi điều khiển đọc tín hiệu từ cảm biến thông qua giao diện ngoại vi. Các tín hiệu thô sau đó được đưa qua khối tiền xử lý. Nhằm tối ưu hóa chu kỳ lệnh cho vi điều khiển không hỗ trợ phần cứng xử lý dấu phẩy động, thuật toán lọc làm trơn theo hàm mũ được triển khai hoàn toàn bằng phép toán số nguyên. Dữ liệu sau khi lọc được đối chiếu với các dải khoảng cách tiêu chuẩn để phân loại hệ thống thành bốn cấp độ: Nguy hiểm, Cảnh báo, Chú ý và An toàn. Chi tiết luồng xử lý tín hiệu được minh họa tại Hình 3.7.



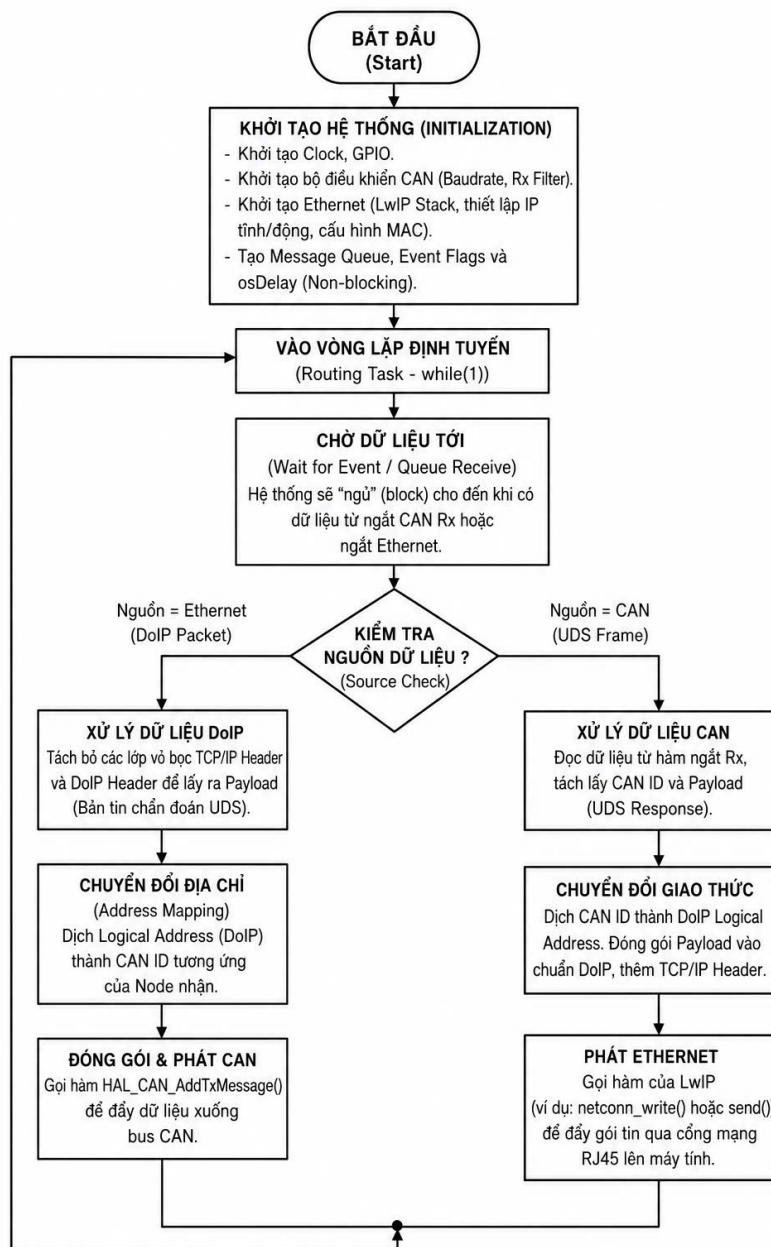
Hình 3.7: Lưu đồ chi tiết thuật toán lọc số học và phân loại ngưỡng cảnh báo

Giai đoạn thứ hai là đóng gói và truyền bản tin CAN. Dữ liệu sau khi phân loại trạng thái được gán mã định danh tương ứng. Thông tin này được đóng gói vào cấu trúc khung tin tiêu chuẩn với tải trọng tối đa 8 bytes. Trước khi đẩy dữ liệu ra bus mạng thông qua API điều khiển ngoại vi, hệ thống sẽ kiểm tra trạng thái của các hộp thư truyền. Nếu không có hộp thư nào trống do nghẽn mạng, vi điều khiển sẽ chủ động hủy yêu cầu gửi cũ để nhường chỗ cho bản tin mang trạng thái cập nhật nhất, đảm bảo tính thời gian thực của dữ liệu chẩn đoán.

3.3.4 Thuật toán định tuyến Gateway (STM32F4)

Đây là chức năng cốt lõi của hệ thống, đảm nhận nhiệm vụ thông dịch và định tuyến dữ liệu giữa hai nền tảng giao thức có sự khác biệt lớn về cấu trúc cũng như băng thông truyền dẫn. Dưới sự điều phối của hệ điều hành thời gian thực FreeRTOS, quá trình định tuyến hai chiều từ CAN sang Ethernet DoIP được thực thi qua ba giai đoạn chính:

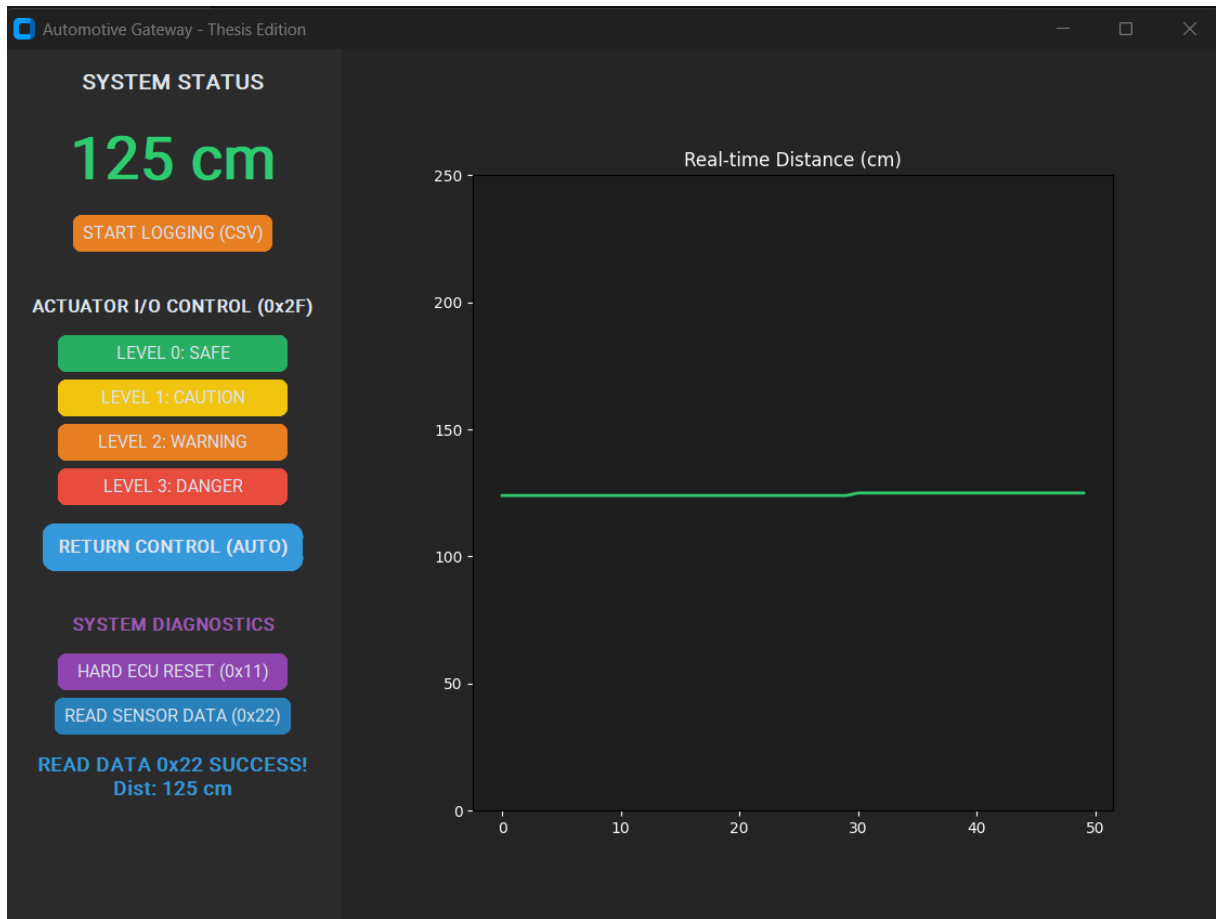
1. Tiếp nhận và bóc tách: Thuật toán khởi tạo bộ lọc phần cứng tại tầng vật lý ở chế độ danh sách định danh. Hệ thống được thiết lập để chỉ chấp nhận các khung tin mang mã định danh hợp lệ đã được quy định trước, bao gồm dữ liệu đo lường định kỳ và phản hồi chẩn đoán UDS. Việc chặn lọc này giúp loại bỏ hoàn toàn các gói tin không liên quan ngay từ mức phần cứng, tối ưu hóa tài nguyên và giảm thiểu gánh nặng xử lý ngắt cho vi điều khiển trung tâm. Ngay khi một khung tin hợp lệ đi qua bộ đệm nhận, thuật toán tiến hành bóc tách phần tải trọng chứa thông tin cốt lõi ra khỏi các trường điều khiển của chuẩn CAN.
2. Chuyển đổi giao thức: Dữ liệu sau khi bóc tách được chuyển vào hàng đợi thông điệp để xử lý bất đồng bộ giữa các luồng tác vụ. Tại đây, dữ liệu được định dạng lại và bổ sung các trường thông tin mào đầu bắt buộc theo tiêu chuẩn giao thức DoIP (ISO 13400). Các trường này bao gồm: Phiên bản giao thức, Loại tải trọng và Chiều dài thông điệp, nhằm đảm bảo tính đồng bộ và toàn vẹn khi định tuyến sang môi trường mạng mới.
3. Đóng gói và truyền tải: Bản tin DoIP sau khi hoàn thiện cấu trúc sẽ được chuyển giao xuống ngăn xếp mạng LwIP để tiếp tục đóng gói thành khung tin UDP/IPv4 tiêu chuẩn. Việc sử dụng UDP giúp tối ưu hóa thời gian thực thi lệnh và đáp ứng yêu cầu độ trễ thấp của hệ thống. Cuối cùng, luồng dữ liệu số được xuất ra giao diện truyền thông trung gian RMII và truyền qua vi mạch vật lý PHY để đưa lên mạng LAN.



Hình 3.8: Lưu đồ thuật toán định tuyến chuyển đổi giao thức CAN - Ethernet DoIP tại Gateway

3.4 Thiết kế giao diện người dùng và chức năng

Giao diện phần mềm giám sát được phát triển dựa trên thư viện customtkinter, ứng dụng chủ đề tối nhằm giảm thiểu sự điều tiết của mắt và tối ưu hóa thị giác cho kỹ sư trong các phiên làm việc kéo dài. Bố cục tổng thể được phân chia khoa học thành các khu vực chức năng độc lập, hỗ trợ quá trình theo dõi và điều khiển thời gian thực.



Hình 3.9: Giao diện phần mềm chẩn đoán UDS Client trên nền tảng PC

Vai trò và luồng thao tác của từng thành phần trên giao diện được quy định thành bốn khu vực chính. Thứ nhất, khu vực trạng thái hệ thống bao gồm màn hình hiển thị trung tâm nhằm trực quan hóa giá trị khoảng cách đo được từ cảm biến (đơn vị cm). Màu sắc cảnh báo tại đây được thiết kế thay đổi linh hoạt tương ứng với các ngưỡng an toàn đã lập trình sẵn. Bên cạnh đó, nút START LOGGING (CSV) đảm nhiệm việc kích hoạt luồng ghi toàn bộ dữ liệu giám sát hiện tại vào tệp tin để phục vụ nhu cầu lưu trữ, trích xuất báo cáo và phân tích ngoại tuyến sau khi kết thúc chu trình chẩn đoán.

Thứ hai, khu vực điều khiển cơ cấu chấp hành tích hợp trực tiếp dịch vụ 0x2F thông qua các nút bấm tương ứng với bốn mức độ cảnh báo định sẵn: An toàn, Chú ý, Cảnh báo và Nguy hiểm. Tính năng này cho phép kỹ sư phát lệnh UDS để giả lập chủ động các trạng thái môi trường, ép vi điều khiển kích hoạt cơ cấu chấp hành mà không cần can thiệp thay đổi khoảng cách vật lý. Nhằm linh hoạt trong quá trình kiểm thử, nút RETURN CONTROL (AUTO) được bố trí để hủy bỏ các lệnh điều khiển giả lập, thu hồi quyền điều khiển và ép hệ thống quay trở lại chế độ cập nhật dữ liệu tự động dựa trên tín hiệu cảm biến thực tế.

Thứ ba, khu vực chẩn đoán hệ thống cung cấp các công cụ tương tác sâu. Nút HARD ECU RESET (0x11) dùng để phát đi bản tin xuống Gateway, yêu cầu thiết bị đầu cuối thực thi tiến trình khởi động lại toàn diện. Nút READ SENSOR DATA (0x22) cho phép chủ động truy xuất các tham số định danh hoặc dữ liệu trạng thái từ node mạng. Góc dưới cùng của khu vực này là dòng trạng thái báo cáo, đóng vai trò hiển thị tức thời kết quả phản hồi tích cực hoặc tiêu cực của các bản tin UDS ngay khi nhận được luồng dữ liệu trả về.

Cuối cùng, khu vực trực quan hóa dữ liệu ứng dụng thư viện vẽ đồ thị để tái hiện quỹ đạo biến thiên của khoảng cách theo thời gian thực. Với trục hoành là số thứ tự chu kỳ lấy mẫu (từ 0 đến 50) và trục tung là biên độ khoảng cách (0 đến 250 cm), biểu đồ cung cấp góc nhìn tổng quan giúp kỹ sư theo dõi trạng thái mạng chính xác.

3.5 Cấu trúc bản tin chẩn đoán UDS và DoIP

Trong hệ thống chẩn đoán phân tán giữa phần mềm trên máy tính và các vi điều khiển, việc chuẩn hóa cấu trúc dữ liệu là yếu tố bắt buộc để đảm bảo tính đồng bộ. Hệ thống sử dụng kết hợp tiêu chuẩn DoIP (ISO 13400) [2] cho lớp truyền tải Ethernet và UDS (ISO 14229) [1] cho lớp ứng dụng chẩn đoán.

3.5.1 Cấu trúc chung của khung truyền DoIP

Mọi gói tin giao tiếp giữa phần mềm máy tính và Gateway đều phải tuân thủ cấu trúc Header DoIP tiêu chuẩn đã được phân tích chi tiết tại Bảng 2.2 (Chương 2). Một khung truyền DoIP cơ bản luôn bao gồm 8 byte Header, theo sau là phần Payload chứa dữ liệu thực tế. Tùy thuộc vào giá trị của trường Payload Type được chỉ định trong Header, nội dung Payload phía sau sẽ được Gateway thông dịch thành các bản tin chẩn đoán UDS hoặc các tín hiệu quản lý mạng tương ứng.

3.5.2 Đóng gói bản tin ứng dụng UDS

Khi trường Payload Type trong Header DoIP được xác định là 0x8001, phần Payload sẽ bao gồm 4 byte địa chỉ logic phân tuyến theo tiêu chuẩn DoIP: 2 byte Source Address đại diện cho thiết bị kiểm thử và 2 byte Target Address định danh node mạng ECU. Việc sử dụng cơ chế địa chỉ này giúp Gateway định tuyến chéo luồng dữ liệu từ mạng Ethernet sang các node trên bus CAN tương ứng. Ngay sau 4 byte địa chỉ này là cấu trúc khung chuẩn UDS. Các lệnh điều khiển từ giao diện phần mềm được ánh xạ chặt chẽ thành các mã SID cụ thể của chuẩn UDS.

Đối với lệnh khởi động lại ECU (Service 0x11), bản tin UDS được đóng gói bao gồm mã SID 0x11 và tham số Sub-function 0x01 (Hard Reset) hoặc 0x03 (Soft Reset). ECU đích sau khi nhận lệnh sẽ trả về mã phản hồi tích cực 0x51 trước khi khởi động lại.

Đối với lệnh đọc dữ liệu cảm biến (Service 0x22), cấu trúc yêu cầu bao gồm mã SID 0x22 kèm theo 2 byte DID để chỉ định vùng nhớ chứa thông số cần truy xuất. Dữ liệu sau đó được ECU đóng gói vào bản tin phản hồi với mã SID 0x62.

Đối với lệnh điều khiển cơ cấu chấp hành (Service 0x2F), chức năng này được sử dụng để giả lập các cấp độ cảnh báo của hệ thống. Bản tin mang theo mã SID 0x2F, mã DID của cơ cấu chấp hành và byte trạng thái điều khiển nhằm ghi đè lên nguyên lý hoạt động tự động của thiết bị.

3.5.3 Cơ chế tinh tiến giao thức tại Gateway

Gateway đóng vai trò là cầu nối dịch thuật trung tâm, thực thi quá trình tinh tiến hai chiều giữa mạng Ethernet và CAN dưới sự điều phối của hệ điều hành FreeRTOS. Khối chức năng này đảm bảo luồng dữ liệu được xử lý liên tục và không bị mất mát khi nhiều thiết bị giao tiếp đồng thời.

Theo chiều từ phần mềm máy tính xuống node mạng, khi nhận gói tin UDP qua cổng Ethernet, ngăn xếp mạng LwIP chuyển giao dữ liệu cho tác vụ định tuyến. Gateway phân tích 8 byte Header DoIP, kiểm tra tính hợp lệ của Protocol Version và Payload Type. Nếu xác thực thành công, thiết bị bóc tách Header và 4 byte địa chỉ logic để trích xuất nguyên bản UDS Payload. Khối dữ liệu này được đưa vào hàng đợi thông điệp. Tác vụ truyền dẫn CAN sau đó lấy dữ liệu, đóng gói vào Data Frame CAN tiêu chuẩn và phát lên bus mạng với Request CAN ID mặc định là 0x7E0.

Theo chiều ngược lại, khi ECU xử lý xong lệnh và trả về kết quả với Response CAN ID là 0x7E8, ngắt phần cứng được kích hoạt để Gateway thu nhận Data Frame này. Vì điều khiển tiến hành xử lý bằng cách tính toán lại độ dài để khởi tạo 8 byte Header DoIP mới, đồng thời bọc thêm 4 byte địa chỉ logic tương ứng. Cuối cùng, khối dữ liệu hoàn chỉnh được đẩy ngược qua socket UDP lên phần mềm máy tính để cập nhật trạng thái hiển thị trên giao diện giám sát, khép kín quy trình tinh tiến dữ liệu.

3.6 Phương pháp đo lường và đánh giá hiệu năng hệ thống

Để chứng minh tính khả thi và độ tin cậy của khối Gateway trong môi trường mạng ô tô thực tế [8], hệ thống không chỉ được kiểm thử về mặt chức năng mà còn phải trải qua các phép đo lường hiệu năng khắt khe. Hai chỉ số quan trọng nhất được đánh giá bao gồm: độ trễ định tuyến và mức độ chiếm dụng tài nguyên CPU.

3.6.1 Kỹ thuật đo lường độ trễ cấp vi mô thông qua thanh ghi DWT

Độ trễ định tuyến là khoảng thời gian kể từ khi vi điều khiển nhận tín hiệu ngắt báo có dữ liệu đầu vào đến khi luồng dữ liệu được bóc tách, chuyển đổi và đẩy ra bộ đệm truyền tải. Để đo lường thông số này với độ phân giải micro-giây mà không tiêu tốn tài nguyên hệ điều hành, hệ thống khai thác thanh ghi đếm chu kỳ DWT được tích hợp phần cứng trong lõi ARM Cortex-M4 của Gateway.

Quá trình đo lường được thực thi bằng cách chen các điểm đánh dấu thời gian trực tiếp vào luồng lệnh. Khi ngắt nhận dữ liệu kích hoạt, hệ thống trích xuất giá trị chu kỳ xung nhịp từ DWT. Sau khi hoàn tất định tuyến và trước khi gọi lệnh phát dữ liệu, giá trị chu kỳ lần thứ hai được trích xuất. Độ trễ thực tế được tính toán qua chênh lệch số lượng chu kỳ lệnh giữa hai điểm, kết hợp với tần số dao động của vi điều khiển, giúp loại bỏ hoàn toàn sai số từ môi trường truyền dẫn bên ngoài.

3.6.2 Đánh giá độ trễ toàn trình dưới điều kiện tải cao

Bên cạnh độ trễ cấp vi mô bên trong chip, độ trễ vĩ mô của toàn bộ hệ thống được đánh giá khách quan thông qua công cụ phân tích mạng Wireshark. Kịch bản kiểm thử được thiết lập bằng cách yêu cầu phần mềm trên máy tính liên tục phát đi các gói tin yêu cầu chẩn đoán theo chuẩn DoIP. Phần mềm Wireshark sẽ ghi nhận dấu thời gian tại thời điểm gói tin rời khỏi máy tính và đối chiếu với dấu thời gian khi nhận được gói tin phản hồi từ các node CAN truyền ngược về.

Để kiểm chứng độ ổn định và giới hạn của hệ thống, phép đo này được thực hiện trong môi trường kiểm thử chịu tải. Kỹ thuật mô phỏng phần mềm được áp dụng nhằm tạo ra từ 5 đến 10 node giả lập, liên tục phát sóng tràn ngập các khung tin CAN mang đa dạng mã định danh vào Gateway. Việc kết hợp phép đo độ trễ toàn trình dưới tải trọng mạng lớn cho phép đánh giá chính xác năng lực phân bổ tài nguyên của bộ lập lịch FreeRTOS; qua đó, kiểm chứng khả năng duy trì thời gian đáp ứng mang tính xác định của hệ thống ngay cả khi xảy ra hiện tượng bão hòa mạng.

CHƯƠNG 4. KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ

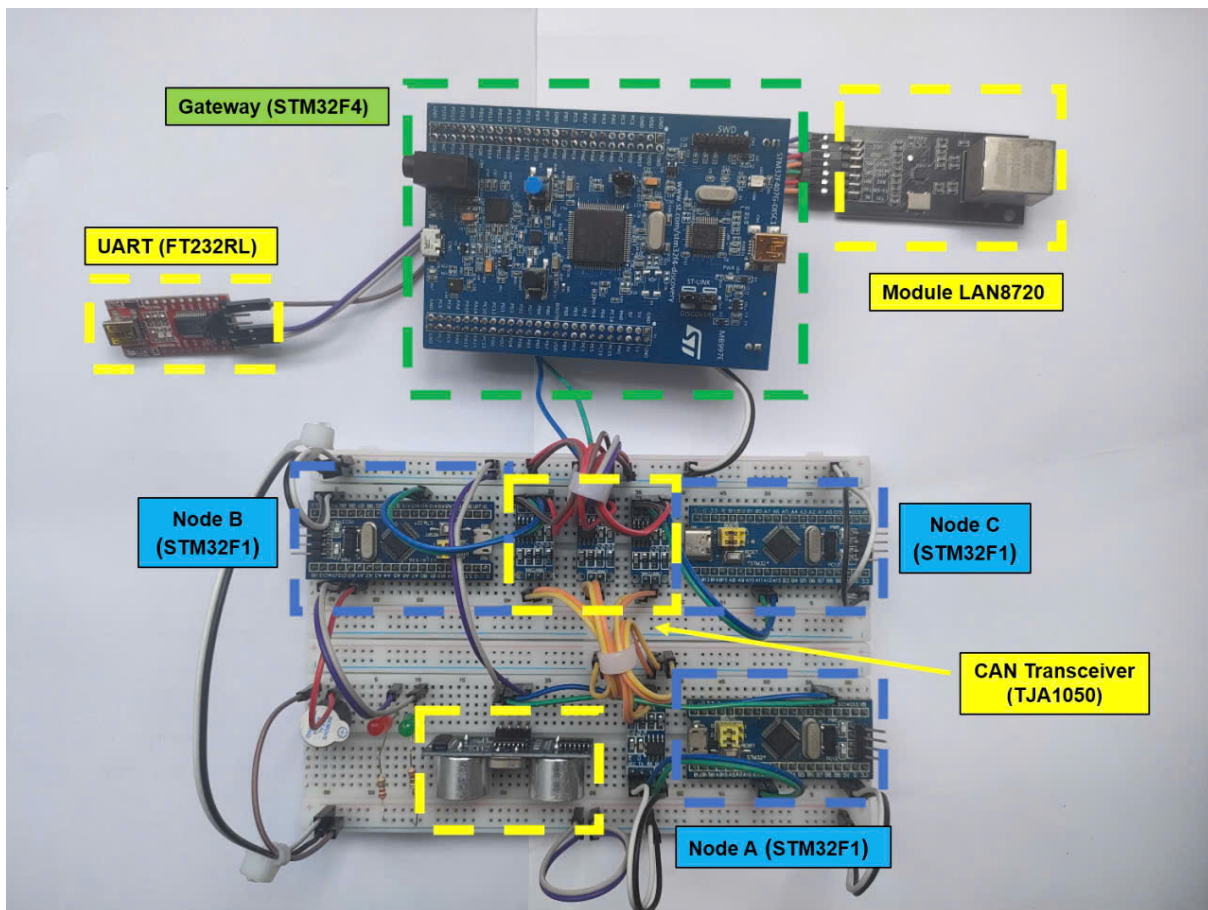
Để kiểm chứng tính đúng đắn của giải thuật và đánh giá hiệu năng thực tế của hệ thống Gateway tinh tiến giao thức DoIP/CAN, quá trình thực nghiệm được tiến hành trên mô hình phần cứng vật lý. Môi trường kiểm thử được xây dựng bao gồm sự phối hợp nhịp nhàng giữa các vi điều khiển, giao diện giám sát và phần mềm phân tích mạng chuyên dụng.

4.1 Môi trường và thiết bị thực nghiệm

4.1.1 Cấu hình phần cứng

Mô hình thực nghiệm được kết nối theo dạng mạng cục bộ, trong đó Gateway đóng vai trò trung tâm xử lý và chuyển tiếp dữ liệu giữa môi trường máy tính cá nhân và bus mạng công nghiệp:

- Thiết bị Tester (máy tính cá nhân): Đóng vai trò là thiết bị chẩn đoán trung tâm, chạy phần mềm GUI để gửi lệnh chẩn đoán và thu thập dữ liệu. Máy tính được kết nối vật lý với mạch Gateway thông qua cáp mạng LAN chuẩn Ethernet (RJ45).
- Khối Gateway (STM32F4): Sử dụng vi điều khiển dòng STM32F4 có tích hợp bộ điều khiển Ethernet MAC. Mạch được gắn thêm module thu phát vật lý PHY (LAN8720) để xử lý tín hiệu mạng, đồng thời kết nối với module CAN Transceiver (TJA1050) để giao tiếp với mạng CAN. Khối này kết nối với máy tính qua cổng gỡ lỗi thông qua module chuyển đổi UART-USB (FT232RL) để xuất các System Logs trong quá trình hoạt động.
- Mạng Node Chấp hành (STM32F1): Để mô phỏng một mạng lưới ô tô đa điểm thực tế, hệ thống sử dụng ba vi điều khiển STM32F1 (được đánh nhãn Node A, Node B, Node C) hoạt động độc lập nhưng giao tiếp song song trên cùng một đường bus mạng. Trong kiến trúc này, Node A được giao nhiệm vụ giao tiếp với cảm biến siêu âm (HC-SR04) để liên tục thu thập tham số môi trường; các Node còn lại chịu trách nhiệm điều khiển cơ cấu chấp hành và đồng thời đóng vai trò là các Dummy Node, phục vụ trực tiếp cho các kịch bản kiểm thử chịu tải và đánh giá năng lực giải quyết xung đột của khối Gateway.



Hình 4.1: Mô hình kết nối phần cứng thực nghiệm hệ thống Gateway DoIP/CAN

4.1.2 Công cụ phần mềm

Quá trình lập trình, kiểm thử và vận hành hệ thống được thực thi và giám sát thông qua các công cụ phần mềm chuyên dụng sau:

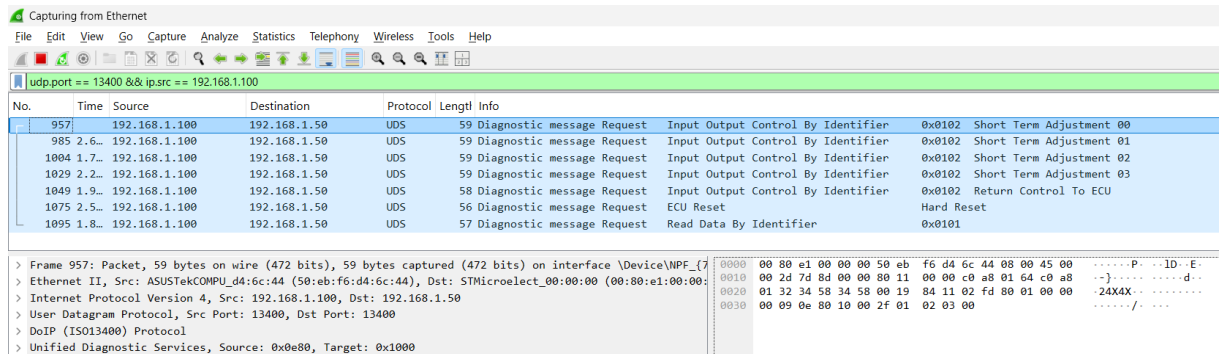
- Phần mềm chẩn đoán Tester GUI / Hercules: Ứng dụng Desktop phục vụ đóng gói lệnh chẩn đoán UDS vào tải trọng DoIP, truyền dẫn qua giao thức UDP (tại cổng 13400) và trực quan hóa dữ liệu phản hồi từ mạng Node theo thời gian thực.
- Phần mềm Wireshark: Công cụ phân tích giao thức mạng độc lập, được triển khai để thám sát và trích xuất các gói tin tại cổng 13400 (với bộ lọc chuyên biệt như `udp.port == 13400 && ip.src`). Công cụ này đóng vai trò tham chiếu khách quan nhằm đối chiếu và xác minh tính tuân thủ tiêu chuẩn ISO 13400 của các bản tin do Gateway định tuyến.
- Keil MDK-ARM & STM32CubeMX: Nền tảng môi trường phát triển tích hợp (IDE) phục vụ khởi tạo ngoại vi, biên dịch mã nguồn và gỡ lỗi trực tiếp luồng thực thi Firmware, kết hợp cùng phần mềm Terminal để thu thập System Logs qua UART.

4.2 Kiểm thử chức năng và chuẩn hóa giao thức

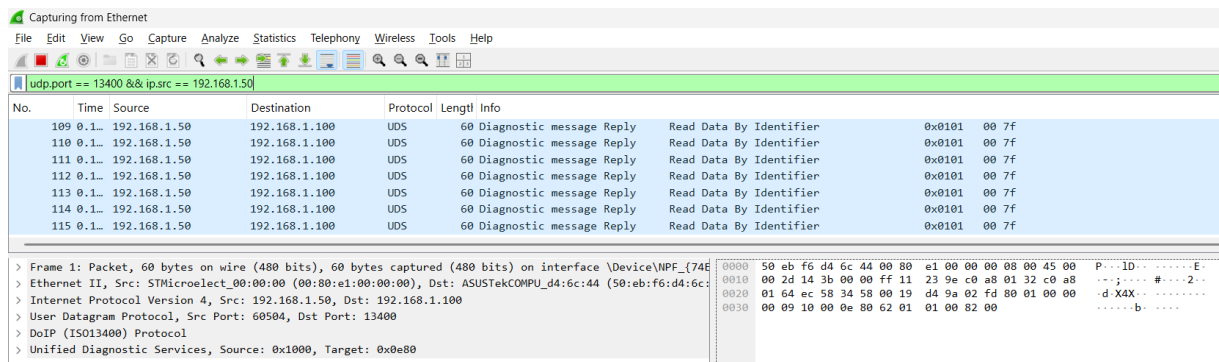
Quá trình kiểm thử chức năng được triển khai để xác minh tính chính xác trong việc đóng/mở gói tin DoIP, định tuyến thông điệp UDS qua lại giữa mạng Ethernet và CAN.

4.2.1 Xác minh gói tin trên Wireshark

Việc theo dõi lưu lượng mạng bằng Wireshark giúp kiểm chứng cấu trúc bản tin truyền dẫn theo tiêu chuẩn ISO 13400.



Hình 4.2: Trích xuất gói tin Request từ phía máy tính Tester qua bộ lọc Wireshark



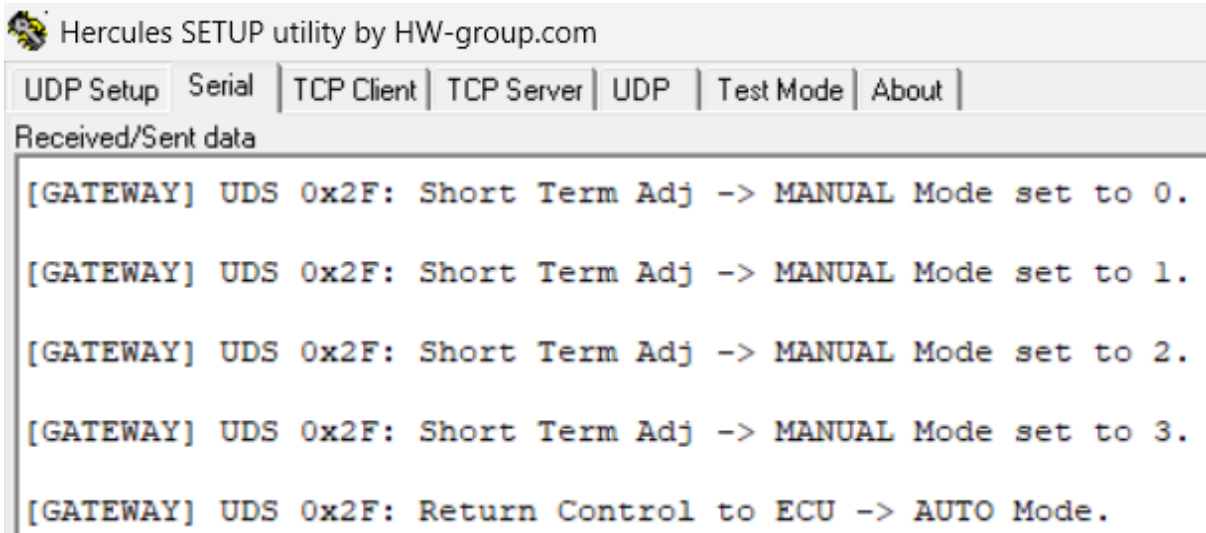
Hình 4.3: Gói tin Reply phản hồi từ Gateway về máy tính Tester

Quan sát Hình 4.2 và Hình 4.3 cho thấy quá trình trao đổi dữ liệu hoàn toàn khớp với kịch bản định tuyến. Các luồng DoIP Request từ Tester (IP 192.168.1.100, logic 0x0E80) đều được hướng chính xác đến Gateway (IP 192.168.1.50, logic 0x1000) qua cổng UDP 13400. Ở chiều ngược lại, các gói DoIP Reply cũng được định tuyến trả về đúng địa chỉ IP nguồn. Toàn bộ lưu lượng mạng đều có cấu trúc Payload Type 0x8001 với kích thước nhất quán, không xảy ra hiện tượng phân mảnh hay rút gói. Kết quả này minh chứng sự ổn định của ngăn xếp LwIP, đảm bảo toàn vẹn dữ liệu và tuân thủ nghiêm ngặt chuẩn chẩn đoán.

4.2.2 Kiểm thử các dịch vụ chẩn đoán UDS cốt lõi

Các dịch vụ UDS tiêu biểu đã được tích hợp và kiểm chứng thành công thông qua log hệ thống xuất ra từ cổng UART:

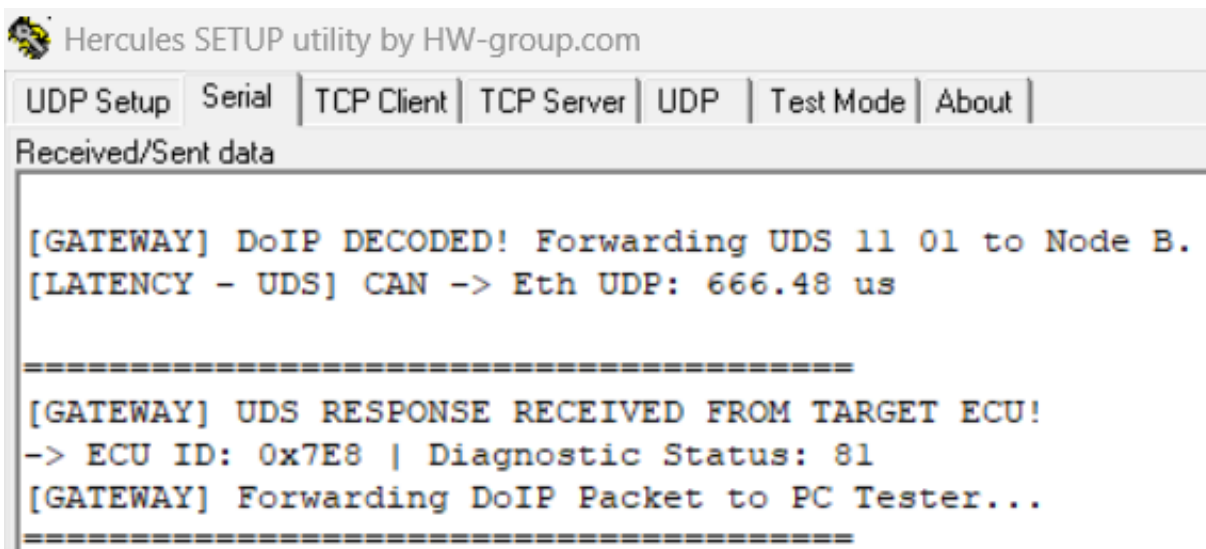
1. Dịch vụ Điều khiển cơ cấu chấp hành (Service 0x2F):



```
Hercules SETUP utility by HW-group.com
UDP Setup | Serial | TCP Client | TCP Server | UDP | Test Mode | About |
Received/Sent data
[GATEWAY] UDS 0x2F: Short Term Adj -> MANUAL Mode set to 0.
[GATEWAY] UDS 0x2F: Short Term Adj -> MANUAL Mode set to 1.
[GATEWAY] UDS 0x2F: Short Term Adj -> MANUAL Mode set to 2.
[GATEWAY] UDS 0x2F: Short Term Adj -> MANUAL Mode set to 3.
[GATEWAY] UDS 0x2F: Return Control to ECU -> AUTO Mode.
```

Hình 4.4: Log UART minh họa quá trình thực thi dịch vụ 0x2F với chức năng Short Term Adjustment, thay đổi các mức tín hiệu chấp hành tuần tự từ Mode 0 đến Mode 3 và lệnh trả về chế độ tự động AUTO Mode

2. Dịch vụ Khởi động lại ECU (Service 0x11):

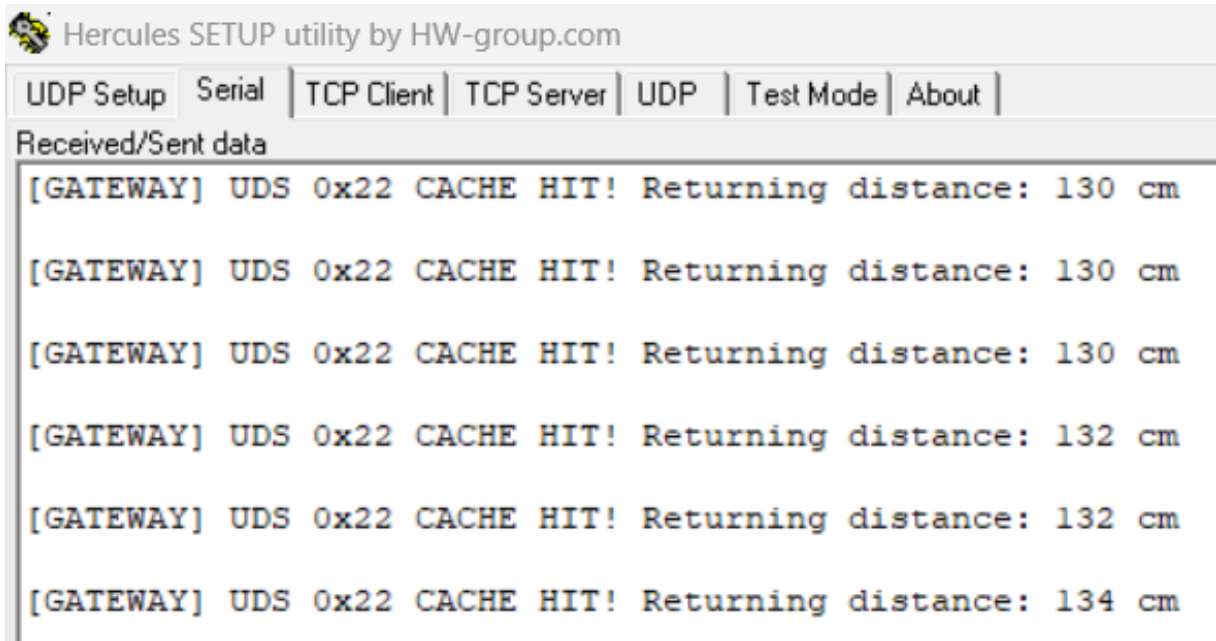


```
Hercules SETUP utility by HW-group.com
UDP Setup | Serial | TCP Client | TCP Server | UDP | Test Mode | About |
Received/Sent data
[GATEWAY] DoIP DECODED! Forwarding UDS 11 01 to Node B.
[LATENCY - UDS] CAN -> Eth UDP: 666.48 us

=====
[GATEWAY] UDS RESPONSE RECEIVED FROM TARGET ECU!
-> ECU ID: 0x7E8 | Diagnostic Status: 81
[GATEWAY] Forwarding DoIP Packet to PC Tester...
=====
```

Hình 4.5: Log UART ghi nhận toàn bộ quá trình xử lý dịch vụ 0x11, bao gồm giải mã gói tin DoIP, chuyển tiếp lệnh khởi động lại xuống Node mạng và xử lý gói tin phản hồi mang trạng thái chẩn đoán từ ECU đích

3. Dịch vụ Đọc dữ liệu cảm biến (Service 0x22):

The screenshot shows the Hercules SETUP utility interface. At the top, there's a title bar 'Hercules SETUP utility by HW-group.com'. Below it is a menu bar with tabs: 'UDP Setup', 'Serial', 'TCP Client', 'TCP Server', 'UDP', 'Test Mode', and 'About'. The 'Serial' tab is selected. Below the menu bar is a section titled 'Received/Sent data'. Inside this section, there is a log of data received over the serial port. The log consists of six lines, each starting with '[GATEWAY]' followed by 'UDS 0x22 CACHE HIT! Returning distance:'. The distance values are 130 cm, 130 cm, 130 cm, 132 cm, 132 cm, and 134 cm respectively.

```
[GATEWAY] UDS 0x22 CACHE HIT! Returning distance: 130 cm
[GATEWAY] UDS 0x22 CACHE HIT! Returning distance: 130 cm
[GATEWAY] UDS 0x22 CACHE HIT! Returning distance: 130 cm
[GATEWAY] UDS 0x22 CACHE HIT! Returning distance: 132 cm
[GATEWAY] UDS 0x22 CACHE HIT! Returning distance: 132 cm
[GATEWAY] UDS 0x22 CACHE HIT! Returning distance: 134 cm
```

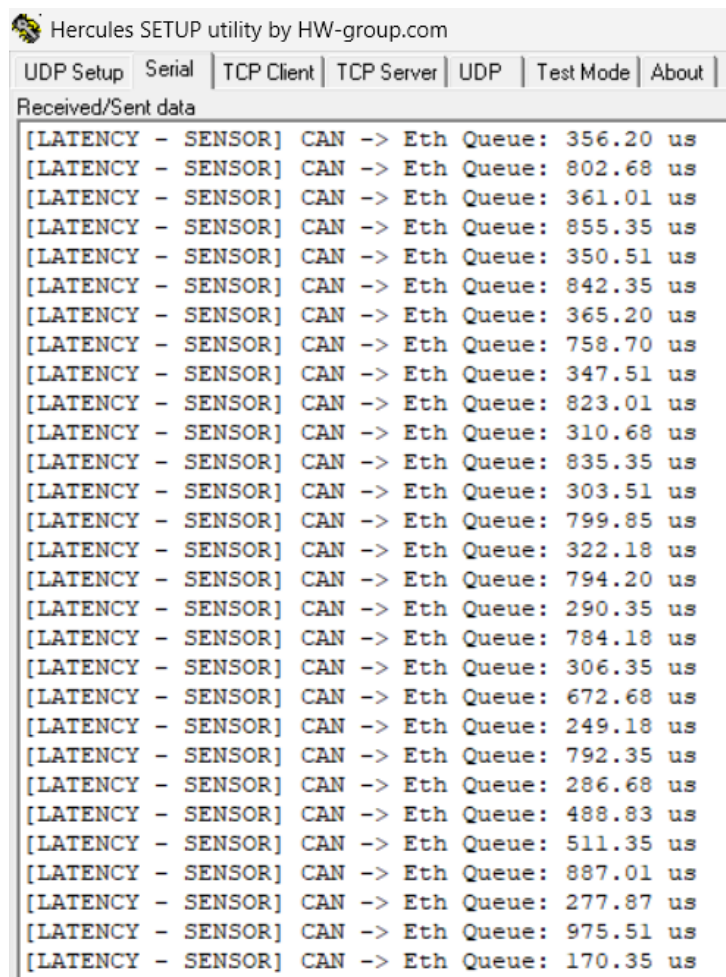
Hình 4.6: Log UART thể hiện kết quả thực thi dịch vụ 0x22 để truy vấn dữ liệu cảm biến siêu âm, minh họa cơ chế tối ưu thông qua Cache Hit giúp trả về tức thời các giá trị khoảng cách đo lường như 130 cm và 132 cm

4.3 Đánh giá hiệu năng độ trễ hệ thống

4.3.1 Kết quả đo lường độ trễ định tuyến cấp vi mô

Sử dụng bộ đếm chu kỳ phần cứng DWT trên lõi ARM Cortex-M4 của vi điều khiển STM32F4 Gateway (168 MHz), thời gian xử lý nội bộ được trích xuất trực tiếp qua luồng log hệ thống giao tiếp UART. Việc trích xuất này ghi nhận chính xác thời điểm một khung dữ liệu chẩn đoán hoàn tất quá trình tịnh tiến từ chuẩn CAN sang Ethernet (tác vụ đẩy vào hàng đợi Eth Queue). Các giá trị đo đếm được hệ thống tự động quy đổi ra đơn vị micro-giây (μs) để thuận tiện cho việc đánh giá độ trễ.

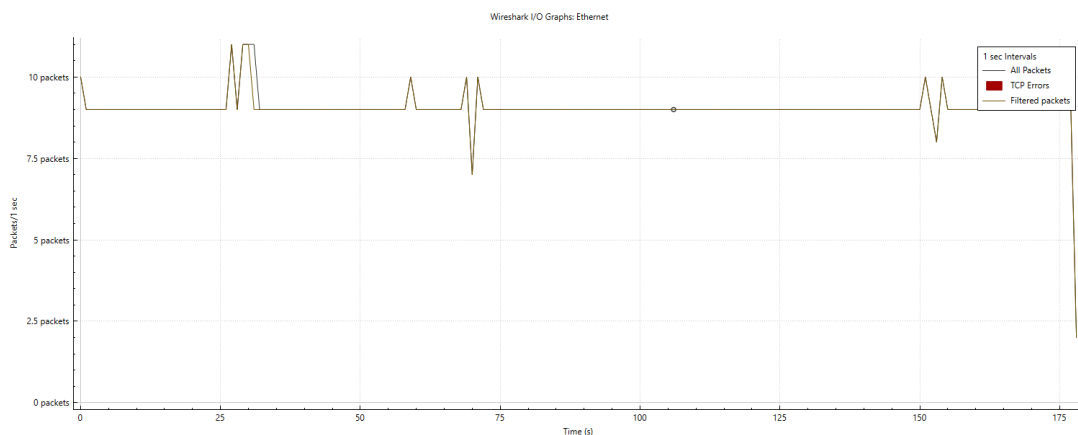
Quá trình thực nghiệm được tiến hành bằng cách gửi liên tục các yêu cầu truy vấn dữ liệu từ cảm biến siêu âm và ghi nhận lại thời gian phản hồi. Kết quả thực nghiệm chi tiết được thể hiện thông qua phần mềm giám sát ở Hình 4.7. Dữ liệu log cho thấy thời gian chuyển mạch bản tin chẩn đoán đạt khoảng $666.48 \mu s$ đến $726.05 \mu s$, trong khi các tác vụ đẩy hàng đợi dữ liệu cảm biến (CAN $\rightarrow$ Eth Queue) dao động linh hoạt từ $170 \mu s$ đến $975 \mu s$. Các thông số này chứng minh hệ thống đáp ứng xuất sắc tiêu chuẩn thời gian thực khắt khe.



Hình 4.7: Thông số đo lường độ trễ định tuyến micro-latency qua khối DWT

4.3.2 Đánh giá độ trễ toàn trình RTT và kiểm thử độ ổn định

Để khảo sát năng lực duy trì kết nối và xử lý luồng dữ liệu liên tục của hệ thống, công cụ Wireshark IO Graphs được áp dụng để giám sát lưu lượng gói tin Ethernet trong khoảng thời gian xấp xỉ 3 phút (175 giây).

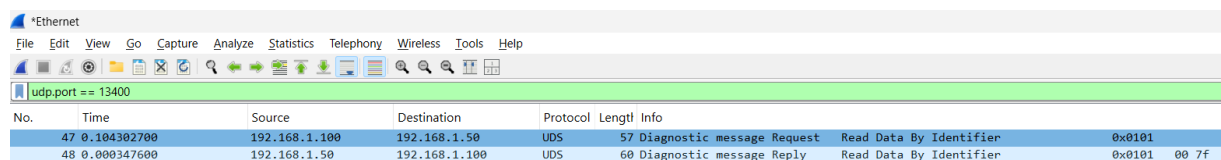


Hình 4.8: Biểu đồ lưu lượng gói tin Ethernet trên Wireshark trong kịch bản kiểm thử hoạt động liên tục

Biểu đồ tại Hình 4.8 cho thấy hệ thống duy trì mức lưu lượng định tuyến trung bình ổn định ở mức 9 đến 11 gói tin/giây. Xuyên suốt quá trình đo kiểm, hệ thống phần mềm chạy nền tảng FreeRTOS chứng minh được tính bền bỉ, đồ thị không có những khoảng sụt giảm về 0 và không ghi nhận hiện tượng rớt gói tin hay đứt gãy luồng giao tiếp giữa mạng IP và CAN bus.

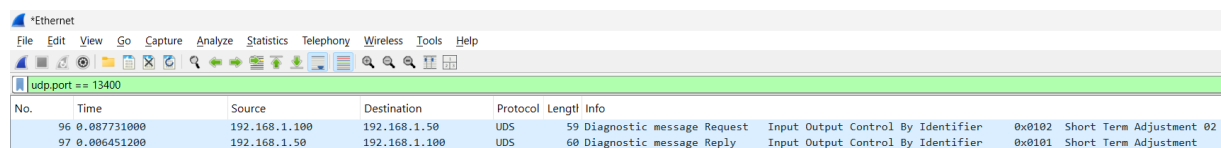
Bên cạnh việc phân tích thông lượng định tuyến, một tiêu chí định lượng quan trọng để đánh giá hiệu năng Gateway là độ trễ toàn trình RTT. RTT được tính từ thời điểm giao diện điều khiển phát đi bản tin yêu cầu chẩn đoán Request đến khi nhận được bản tin phản hồi Reply từ node mạng thông qua Gateway.

Để làm rõ cơ sở của các số liệu thống kê trong Bảng 4.1, các Hình 4.9 (a, b, c) lần lượt trích xuất các bằng chứng đo lường thực tế từ phần mềm Wireshark đối với từng nhóm dịch vụ UDS đặc trưng.



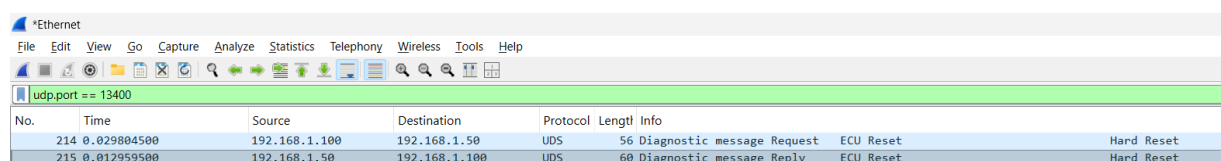
No.	Time	Source	Destination	Protocol	Length	Info
47	0.104302700	192.168.1.100	192.168.1.50	UDS	57	Diagnostic message Request Read Data By Identifier 0x0101
48	0.000347600	192.168.1.50	192.168.1.100	UDS	60	Diagnostic message Reply Read Data By Identifier 0x0101 00 7f

(a) Độ trễ Min RTT (0.35 ms) với lệnh Read Data By Identifier (0x22)



No.	Time	Source	Destination	Protocol	Length	Info
96	0.087731000	192.168.1.100	192.168.1.50	UDS	59	Diagnostic message Request Input Output Control By Identifier 0x0102 Short Term Adjustment 02
97	0.006451200	192.168.1.50	192.168.1.100	UDS	60	Diagnostic message Reply Input Output Control By Identifier 0x0101 Short Term Adjustment

(b) Độ trễ Average RTT (~6.45 ms) với lệnh Input Output Control (0x2F)



No.	Time	Source	Destination	Protocol	Length	Info
214	0.029804500	192.168.1.100	192.168.1.50	UDS	56	Diagnostic message Request ECU Reset Hard Reset
215	0.012959500	192.168.1.50	192.168.1.100	UDS	60	Diagnostic message Reply ECU Reset Hard Reset

(c) Độ trễ Max RTT (12.96 ms) với lệnh ECU Reset (0x11)

Hình 4.9: Bằng chứng đo lường độ trễ toàn trình RTT của các dịch vụ UDS trên Wireshark

Dựa trên phương pháp đo kiểm này, hệ thống đã thực hiện lấy mẫu với nhiều kịch bản giao tiếp tương ứng với các dịch vụ chẩn đoán UDS khác nhau để đưa ra cái nhìn tổng quan. Kết quả thống kê độ trễ toàn trình được trình bày chi tiết trong Bảng 4.1.

Bảng 4.1: Kết quả định lượng độ trễ toàn trình RTT phân loại theo dịch vụ UDS

Tham số đánh giá	Dịch vụ chẩn đoán UDS Service	Giá trị đo lường (ms)
Độ trễ nhỏ nhất Min RTT	0x22 Read Data By Identifier	0.35
Độ trễ trung bình Average	0x2F Input Output Control	~6.45
Độ trễ lớn nhất Max RTT	0x11 ECU Reset	12.96

Từ số liệu Bảng 4.1, có thể khẳng định độ trễ RTT của hệ thống Gateway phụ thuộc trực tiếp vào đặc thù cấu trúc và khối lượng công việc của từng mã lệnh UDS được yêu cầu:

- Mức trễ tối thiểu (0.35 ms): Đạt được khi thực hiện lệnh đọc dữ liệu 0x22. Mức trễ này cực kỳ thấp do vi điều khiển trung tâm chỉ cần truy xuất các biến trạng thái hoặc vùng nhớ được định nghĩa sẵn và lập tức đóng gói phản hồi mà không mất thời gian thực thi các tác vụ tương tác ngoại vi.
- Mức trễ trung bình (~6.45 ms): Ghi nhận tại các lệnh điều khiển ngoại vi 0x2F. Khoảng thời gian này phản ánh chính xác chi phí xử lý overhead khi hệ thống phải gọi các hàm giao tiếp phần cứng (như thao tác với thư viện HAL để thay đổi trạng thái chân GPIO) kết hợp với thời gian phân xử ưu tiên arbitration trên bus CAN. Sự đồng đều của các mẫu đo (đều đặn ở mức 6.xx ms) minh chứng cho tính ổn định của nền tảng RTOS.
- Mức trễ tối đa (12.96 ms): Xuất hiện tại lệnh 0x11 do bản chất của yêu cầu Hard Reset đòi hỏi vi điều khiển phải thực hiện chuỗi tác vụ phức tạp: lưu trữ ngữ cảnh, phản hồi xác nhận Reply, và tiến hành gọi các lệnh khởi động lại hệ thống.

Nhìn chung, dải độ trễ dao động từ 0.35 ms đến 12.96 ms cho thấy năng lực định tuyến của Gateway hoàn toàn nằm trong quỹ đạo an toàn, đáp ứng xuất sắc các tiêu chuẩn khắt khe về thời gian thực của mạng truyền thông trên ô tô.

4.4 Trích xuất và lưu trữ dữ liệu hệ thống

Bên cạnh việc giám sát trực tiếp trên giao diện phần mềm Tester và công cụ phân tích mạng Wireshark, hệ thống Gateway còn được tích hợp tính năng tự động lưu vết và xuất dữ liệu Data Logging dưới định dạng tệp CSV. Tính năng này đóng vai trò quan trọng trong việc lưu trữ toàn bộ lịch sử giao dịch chẩn đoán, phục vụ trực tiếp cho công tác hậu kiểm, phân tích lỗi và lập biểu đồ đánh giá hiệu năng dài hạn.

1	Timestamp	Direction	UDS Service	Payload/Data	Latency (ms)
2	55:02.2	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
17	55:03.8	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
18	55:03.9	Tx	0x2F	02 FD 80 01 00 00 00 09 0E 80 10 00 2F 01 02 03 00	---
19	55:03.9	Rx	0x2F	02 FD 80 01 00 00 00 08 10 00 0E 80 6F 01 01 03	6.7 ms
20	55:03.9	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
21	55:04.0	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
35	55:05.4	Tx	0x2F	02 FD 80 01 00 00 00 09 0E 80 10 00 2F 01 02 03 01	---
36	55:05.4	Rx	0x2F	02 FD 80 01 00 00 00 08 10 00 0E 80 6F 01 01 03	6.56 ms
37	55:05.5	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
38	55:05.6	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
55	55:07.4	Tx	0x2F	02 FD 80 01 00 00 00 09 0E 80 10 00 2F 01 02 03 02	---
56	55:07.4	Rx	0x2F	02 FD 80 01 00 00 00 08 10 00 0E 80 6F 01 01 03	6.58 ms
57	55:07.4	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
58	55:07.5	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
70	55:08.7	Tx	0x2F	02 FD 80 01 00 00 00 09 0E 80 10 00 2F 01 02 03 03	---
71	55:08.7	Rx	0x2F	02 FD 80 01 00 00 00 08 10 00 0E 80 6F 01 01 03	6.59 ms
72	55:08.7	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
73	55:08.8	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1D	---
87	55:10.2	Tx	0x2F	02 FD 80 01 00 00 00 08 0E 80 10 00 2F 01 02 00	---
88	55:10.2	Rx	0x2F	02 FD 80 01 00 00 00 08 10 00 0E 80 6F 01 01 03	6.33 ms
89	55:10.3	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 1E	---
90	55:10.4	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 20	---
99	55:11.3	Tx	0x22	02 FD 80 01 00 00 00 07 0E 80 10 00 22 01 01	---
100	55:11.3	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 22	0.53 ms
101	55:11.4	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 22	---
102	55:11.5	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 22	---
115	55:12.8	Tx	0x11	02 FD 80 01 00 00 00 06 0E 80 10 00 11 01	---
116	55:12.8	Rx	0x11	02 FD 80 01 00 00 00 06 10 00 0E 80 51 01	7.19 ms
117	55:12.8	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 22	---
118	55:12.9	Rx	0x22	02 FD 80 01 00 00 00 09 10 00 0E 80 62 01 01 00 22	---

Hình 4.10: Dữ liệu chẩn đoán UDS và thông số độ trễ được trích xuất thành file CSV

Như được thể hiện trên Hình 4.10, tệp nhật ký CSV ghi nhận một cách có hệ thống các trường thông tin quan trọng của từng luồng dữ liệu theo thời gian thực, bao gồm:

- **Timestamp:** Dấu thời gian ghi nhận gói tin (với độ phân giải đạt mức mili-giây).
- **Direction:** Hướng truyền dữ liệu (ví dụ: Tx - Lệnh xuất phát từ Tester, Rx - Phản hồi nhận từ Gateway).
- **UDS Service:** Mã lệnh dịch vụ chẩn đoán cốt lõi (như 0x22, 0x2F, 0x11).
- **Payload/Data:** Chuỗi dữ liệu thô Hexadecimal chứa nội dung truy vấn hoặc thông số trả về từ các Node mạng CAN.
- **Latency (ms):** Chỉ số độ trễ toàn trình đo lường được cho từng chu kỳ Request/Reply.

Đặc biệt, thông qua việc đối chiếu dữ liệu trên Hình 4.10, các chu kỳ đo kiểm độ trễ toàn trình RTT của từng dịch vụ UDS đã được trích xuất và làm nổi bật. Sự ngắt quãng có chủ đích của các số thứ tự dòng dữ liệu (ví dụ từ dòng 38 nhảy sang 55, hoặc 73 sang 87) cho thấy trong lúc chờ lệnh can thiệp thủ công từ Tester, hệ thống Gateway vẫn liên tục định tuyến hàng ngàn gói tin dữ liệu nền (đọc cảm biến định kỳ) một cách bền bỉ mà không ghi nhận độ trễ tích lũy hay nghẽn cổ chai.

Việc xuất và lưu trữ thành công các thông số trên chứng minh nền tảng phần mềm giám sát đã được phát triển hoàn thiện, đáp ứng đầy đủ chu trình kiểm thử tự động trong môi trường mạng giao tiếp ô tô hiện đại.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết luận chung

Khóa luận đã hoàn thành mục tiêu thiết kế và xây dựng thành công mô hình Gateway chẩn đoán trên ô tô, ứng dụng các tiêu chuẩn truyền thông công nghiệp hiện đại bao gồm Ethernet (DoIP) và CAN (UDS). Hệ thống hoạt động ổn định, đáp ứng được các yêu cầu về tính thời gian thực và khả năng tịnh tiến giao thức chính xác giữa các mạng dữ liệu khác biệt.

5.1.1 Đóng góp của đề tài

Về mặt kỹ thuật và nghiên cứu thực tiễn, đề tài đã hoàn thành và mang lại những đóng góp cụ thể như sau:

- Thiết kế phần cứng và kiến trúc nhúng: Xây dựng thành công mô hình mạng chẩn đoán vật lý bao gồm thiết bị trung tâm Gateway (STM32F4) và các Node chấp hành (STM32F1) kết nối qua bus CAN. Tích hợp hệ điều hành thời gian thực FreeRTOS để quản lý đa luồng, đảm bảo không xảy ra hiện tượng Packet Loss.
- Định tuyến và tịnh tiến giao thức: Triển khai thành công chu trình khép kín của các dịch vụ chẩn đoán UDS cơ bản (0x11, 0x22, 0x2F) được đóng gói trong khung truyền DoIP (ISO 13400), thực hiện định tuyến hai chiều chính xác giữa mạng CAN và Ethernet.
- Phát triển công cụ giám sát Tester GUI: Lập trình giao diện người dùng bằng Python (CustomTkinter) tích hợp đa luồng, cho phép gửi lệnh chẩn đoán trực tiếp, ghi nhận phản hồi và trực quan hóa dữ liệu cảm biến theo thời gian thực.
- Đánh giá thực nghiệm RTT: Thực hiện đo lường và phân tích chi tiết độ trễ toàn trình RTT của các lệnh chẩn đoán, cung cấp cơ sở dữ liệu thực tiễn về khả năng đáp ứng thời gian thực của hệ thống nhúng trong môi trường chuẩn mạng ô tô.

5.1.2 Kiến thức và kỹ năng đạt được

Thông qua quá trình nghiên cứu và thực hiện đề tài, các kiến thức và kỹ năng chuyên môn đã được củng cố và mở rộng:

- Nâng cao kiến thức nền tảng về Embedded Systems và kiến trúc vi điều khiển, đặc biệt là việc làm chủ dòng vi điều khiển STM32 và thư viện lập trình HAL.

- Hiểu rõ và có khả năng ứng dụng các tiêu chuẩn mạng truyền thông trên ô tô như CAN bus, Ethernet và các giao thức chẩn đoán chuyên sâu UDS, DoIP.
- Nâng cao tư duy thiết kế phần mềm hệ thống, bao gồm kỹ năng lập trình Multithreading, quản lý Queue và đồng bộ hóa tiến trình.
- Hoàn thiện kỹ năng trình bày, tổng hợp tài liệu và soạn thảo báo cáo học thuật chuyên nghiệp với cấu trúc chặt chẽ sử dụng hệ thống LaTeX.

5.2 Hạn chế của đề tài

Mặc dù đã đạt được những kết quả khả quan, hệ thống hiện tại vẫn tồn tại một số hạn chế do giới hạn về thời gian và điều kiện thực nghiệm:

- Về phần cứng: Hệ thống vẫn đang được triển khai trên các Discovery board và testboard với nhiều dây nối jumper, chưa thiết kế thành một PCB hoàn chỉnh, do đó độ nhiễu và tính ổn định trong môi trường công nghiệp thực tế chưa đạt mức tối ưu.
- Về giao thức UDS: Đề tài mới chỉ tập trung cài đặt một số Service chẩn đoán cơ bản nhất, chưa bao phủ toàn bộ bộ tiêu chuẩn ISO 14229 như các dịch vụ nạp lại phần mềm Flashing hay Security Access.
- Về bảo mật: Tính năng Cybersecurity cho mạng xe hơi như mã hóa dữ liệu DoIP qua TLS hay xác thực gói tin CAN chưa được tích hợp.

5.3 Hướng phát triển

Dựa trên những hạn chế đã nêu, đề tài mở ra một số hướng nghiên cứu và phát triển tiếp theo trong tương lai:

- Tối ưu hóa phần cứng: Thiết kế PCB chuyên dụng cho Gateway tích hợp sẵn các IC thu phát Ethernet PHY, CAN Transceiver với thiết kế chống nhiễu điện từ EMI để tăng độ bền bỉ của hệ thống.
- Mở rộng dịch vụ chẩn đoán: Tích hợp thêm tính năng cập nhật phần mềm từ xa OTA thông qua giao thức DoIP và các dịch vụ UDS liên quan đến quản lý bộ nhớ (0x34, 0x36, 0x37).
- Nghiên cứu bảo mật ô tô: Tích hợp các cơ chế bảo mật như TLS cho kết nối Ethernet và SecOC cho mạng CAN để chống lại các cuộc tấn công mạng giả mạo gói tin.

TÀI LIỆU THAM KHẢO

- [1] International Organization for Standardization, “Road vehicles – Unified diagnostic services (UDS) – Part 1: Specification and requirements,” *ISO 14229-1:2020*, 2020.
- [2] International Organization for Standardization, “Road vehicles – Diagnostic communication over Internet Protocol (DoIP) – Part 2: Transport protocol and network layer services,” *ISO 13400-2:2012*, 2012.
- [3] J. Yiu, “The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors,” 3rd ed., Newnes, 2013. ISBN: 978-0124080829.
- [4] R. Barry, “Mastering the FreeRTOS Real Time Kernel,” Real Time Engineers Ltd, 2016.
- [5] K. Matheus and T. Königseder, “Automotive Ethernet,” Cambridge University Press, 2015. ISBN: 978-1107114258.
- [6] STMicroelectronics, “UM1722 User manual: Developing Applications on STM32Cube with RTOS,” Rev. 3, 2018. [Online]. Available: www.st.com.
- [7] T. Schimansky, “CustomTkinter: A modern and customizable python UI-library based on Tkinter,” GitHub Repository, 2023. [Online]. Available: <https://github.com/tomschimansky/customtkinter>.
- [8] H. Singh and M. Sharma, “Automotive Diagnostic Gateway: Architecture and Latency Analysis of CAN to DoIP Protocol Conversion,” *IEEE Transactions on Vehicular Technology*, vol. 68, no. 5, pp. 4310-4320, 2019.
- [9] T. Hoang, D. Le, and C. Pham, “Design and Implementation of an Automotive Diagnostic Gateway based on UDS over CAN and Ethernet,” *2020 IEEE International Conference on Automatics and Electronics*, pp. 112-117, Oct. 2020.
- [10] S. B. Oh and Y. E. Son, “Performance Enhancement of CAN/Ethernet Automotive Gateway with a CAN Data Reduction Algorithm,” *Electronics*, vol. 12, no. 12, p. 2602, Jun. 2023. DOI: 10.3390/electronics12122602.

